ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

CHATBOT PARA ATENCIÓN DE PROCESOS ACADÉMICOS DE GRADO, CASO DE ESTUDIO SUBDECANATO FIS

ANÁLISIS DE HERRAMIENTAS DE CHATBOT, DISEÑO Y CONSTRUCCIÓN DEL BACKEND

TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO REQUISITO PARA LA OBTENCIÓN DEL TÍTULO DE INGENIERO EN SOFTWARE

ANDRÉS DAVID LOZANO ESTRADA

andres.lozano@epn.edu.ec

DIRECTOR: ING. MARÍA MONSERRATE INTRIAGO PAZMIÑO, PhD

monserrate.intriago@epn.edu.ec

DMQ, Febrero 2024

CERTIFICACIONES

Yo, Andrés David Lozano Estrada declaro que el trabajo de integración curricular aquí descrito es de mi autoría; que no ha sido previamente presentado para ningún grado o calificación profesional; y, que he consultado las referencias bibliográficas que se incluyen en este documento.

ANDRÉS DAVID LOZANO ESTRADA

Certifico que el presente trabajo de integración curricular fue desarrollado por Andrés David Lozano Estrada, bajo mi supervisión.

ING. MARÍA MONSERRATE INTRIAGO PAZMIÑO, PhD
DIRECTOR

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el trabajo de integración curricular aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

ANDRÉS DAVID LOZANO ESTRADA

ING. MARÍA MONSERRATE INTRIAGO PAZMIÑO, PhD

DEDICATORIA

A mis queridos padres,

Su apoyo incondicional y su constante preocupación sobre mi bienestar fueron los pilares que me permitieron llegar al final de este camino. Definitivamente, todos mis logros en la vida son el fruto de su profunda comprensión hacia mi carácter y su inquebrantable paciencia frente a mis errores.

Con todo mi cariño, Andrés.

AGRADECIMIENTO

En primer lugar, agradezco profundamente a mis padres, Arturo y Patricia, cuyo esfuerzo diario y sacrificio económico fueron fundamentales en este camino. Su constante respaldo y los recursos que me brindaron hicieron posible este logro.

Agradezco también a mis amigos y compañeros de carrera, con quienes compartí momentos de diversión, estudio y trabajo en equipo que fueron de gran ayuda para superar los desafíos académicos.

Asimismo, quiero reconocer el invaluable aporte de la PhD. Monserrate Intriago, cuya dedicación y orientación fueron fundamentales en el desarrollo de este trabajo.

Por último, agradezco a Paola por su valiosa amistad y por brindarme una perspectiva singular sobre la vida. Gracias por ensenarme a apreciar la belleza de las cosas que por mucho tiempo ignoré.

ÍNDICE DE CONTENIDO

CERTIFICACIONES I
DECLARACIÓN DE AUTORÍA II
DEDICATORIA III
AGRADECIMIENTO IV
ÍNDICE DE CONTENIDO V
RESUMEN VI
ABSTRACT VII
1 INTRODUCCIÓN 1
1.1 Objetivo general 2
1.2 Objetivos específicos 2
1.3 Alcance 3
1.4 Marco teórico 4
2 METODOLOGÍA 11
2.1 Análisis previo de requisitos 11
2.2 Especificación de requisitos 17
2.3 Fase de diseño o planteamiento 19
2.4 Fase de implementación 28
2.5 Fase de pruebas, evaluación y análisis de resultados 33
2.6 Fase de integración 38
3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES 39
3.1 Resultados 40
3.2 Conclusiones 44
3.3 Recomendaciones 45
4 REFERENCIAS BIBLIOGRÁFICAS 47
5 ANEXOS 49
ANEXO I 50
ANEXO II 52
ANEXO III 53
ANEXO IV 54
ANEXO V 55

RESUMEN

Este trabajo de integración curricular aborda el desafío de mejorar la atención estudiantil en la Facultad de Ingeniería de Sistemas (FIS) de la Escuela Politécnica Nacional (EPN). Dada la creciente cantidad de consultas, se identificó la necesidad de una solución eficiente y rápida. En este contexto, se desarrolló el backend de un chatbot centrado en ofrecer respuestas a consultas sobre procesos académicos de grado.

En el primer capítulo, se aborda la problemática específica del subdecanato de la FIS, estableciendo los objetivos que orientan el desarrollo del componente. Se delimita el alcance del trabajo, describiendo las fases ejecutadas y los entregables previstos. Además, se proporciona una visión detallada de los conceptos teóricos y las tecnologías utilizadas durante el desarrollo del componente, sentando las bases para una comprensión integral de la solución propuesta.

En el segundo capítulo, se presenta la metodología de que gestionó el desarrollo del componente. Se describen las fases de desarrollo, incluyendo una fase de análisis y especificación de requisitos, una fase de diseño o planteamiento para proponer una solución, una fase de implementación para construir el backend del chatbot, una fase pruebas para validar la implementación realizada y una fase de integración para incorporar el componente en un entorno que incluye los demás elementos del proyecto.

Finalmente, el tercer capítulo detalla los resultados obtenidos tras el proceso de desarrollo del componente, destacando el estado del backend en el entorno de integración. Además, se brindan conclusiones y recomendaciones en torno a los resultados obtenidos.

PALABRAS CLAVE: Chatbot, API REST, backend, reconocimiento de la intención, patrones de diseño.

ABSTRACT

This curricular integration work addresses the challenge of improving student support at the School of Systems Engineering (FIS) of the National Polytechnic School (EPN). Given the increasing number of inquiries, FIS identify the need for an efficient and quick solution. In this context, the backend of a chatbot focused on providing answers to inquiries about academic degree processes was developed.

The first chapter addresses the specific issues of the FIS subdeanery, establishing the objectives that guide the component's development. The scope of the work is defined, describing the executed phases, and expected deliverables. Additionally, a detailed overview of the theoretical concepts and technologies used during the component's development is provided in this chapter, laying the groundwork for a comprehensive understanding of the proposed solution.

The second chapter presents the methodology that managed the component's development. The development phases are described in this chapter, including an analysis and requirements specification phase, a design phase to propose a solution, an implementation phase to build the chatbot's backend, a testing phase to validate the implementation, and an integration phase to incorporate the component into an environment that includes other project elements.

Finally, the third chapter details the results obtained after the component's development process, highlighting the state of the backend in the integration environment. Additionally, conclusions and recommendations regarding the obtained results are provided.

KEYWORDS: Chatbot, REST API, backend, intent recognition, design patterns.

1 INTRODUCCIÓN

A la fecha actual, la Facultad de Ingeniería de Sistemas (FIS) de la Escuela Politécnica Nacional (EPN) cuenta con más de novecientos estudiantes de grado. Por lo cual, la cantidad de consultas dirigidas a las Secretarias, Decanato, Subdecanato y Coordinaciones de Carrera, ya sea de manera presencial o por medio de correo electrónico, hace que el tiempo de respuesta sea considerable, alcanzando un promedio de 48 horas, tal como se lo detalla en el Anexo I.

En este contexto, actualmente las herramientas de chatbot son ampliamente utilizadas en diferentes organizaciones como una solución de atención inmediata hacia sus clientes o público en general [1]. Así pues, en este trabajo de integración curricular se desarrolla un chatbot para la atención estudiantil enfocada en los procesos académicos de grado gestionados por el subdecanato de la FIS. Específicamente, el componente descrito en este documento se centra en la creación del backend del sistema del chatbot.

En pocas palabras, el componente se enfoca en un análisis detallado de las herramientas de chatbot disponibles comercialmente, el diseño arquitectónico y la implementación del backend de la solución propuesta.

La evaluación de las herramientas existentes resulta fundamental para identificar las características clave que posteriormente se integran en el chatbot. Estas características se documentan de manera estructurada mediante diagramas de diseño UML (Unified Modeling Language), lo que facilita la expansión de la arquitectura del programa a lo largo de las distintas iteraciones. La gestión de las iteraciones se realiza mediante el marco de trabajo Kanban debido a su gran flexibilidad.

La culminación de este proceso alcanza el objetivo de la creación del backend del chatbot a través de la implementación de una API REST (Application Programming Interface), que posibilita la interacción en tiempo real entre los usuarios y el núcleo del sistema. Este último, por su parte, se enfoca en la identificación de las intenciones de los usuarios y en la gestión de respuestas predeterminadas a través una base de conocimiento.

El análisis de las herramientas disponibles permite identificar las funcionalidades esenciales para el chatbot y sirve como punto de partida para la creación de una sólida estructura arquitectónica. Estas características, cuidadosamente seleccionadas, se organizan en documentos de diseño UML, lo que facilita la comprensión del sistema y prepara el camino para la escalabilidad a lo largo de las diversas iteraciones del proyecto.

La implementación del marco de trabajo Kanban demuestra ser adecuada para la gestión eficiente del desarrollo del chatbot. A través de iteraciones bien planificadas, se logra una evolución constante del sistema, permitiendo ajustes y mejoras continuas sin afectar la estabilidad del sistema. La flexibilidad que ofrece Kanban es clave para adaptarse a las necesidades cambiantes del proyecto y asegurar un progreso constante.

En última instancia, la construcción del backend se lleva a cabo mediante la creación de una API REST, que sirve como interfaz fundamental para la interacción entre los usuarios y el sistema. Este componente central se enfoca específicamente en la identificación de las intenciones de los usuarios mediante la adopción de un modelo neuronal especializado en calcular el valor de similitud de los mensajes de los usuarios y la información cargada en la base de conocimiento. Así también, se dedica especial atención al manejo y envío de diversos tipos de respuestas, incluyendo imágenes, texto, enlaces y sugerencias.

En resumen, el desarrollo del componente para el sistema de chatbot en la Facultad de Ingeniería de Sistemas se caracteriza por la aplicación de un enfoque sistemático que abarca desde el análisis de herramientas hasta la implementación del backend. La combinación de un diseño cuidadoso, documentación estructurada y una gestión ágil del proyecto culmina en un sistema robusto y eficiente para la atención de procesos académicos de grado.

1.1 Objetivo general

Proporcionar una solución efectiva, eficiente y personalizable para la atención de procesos académicos de grado de la Facultad de Ingeniería de Sistemas mediante la construcción de una API REST para el reconocimiento de la intención de los usuarios y la generación de respuestas de un chatbot basado en recuperación.

1.2 Objetivos específicos

1. Realizar un análisis de herramientas de chatbot disponibles en la industria, identificando sus arquitecturas, funcionamientos y tecnologías subyacentes para seleccionar las mejores alternativas que permitan ofrecer una solución efectiva ante el caso de estudio de la Facultad de Ingeniería de Sistemas.
2. Contrastar los hallazgos del análisis de herramientas de chatbot con los requerimientos específicos de la Facultad de Ingeniería de Sistemas, para determinar las

características más relevantes y adecuadas que permiten satisfacer la atención de procesos académicos de grado.

3. Diseñar la arquitectura del backend del chatbot mediante la creación de diagramas UML que muestren la estructura y la interacción de componentes para garantizar una implementación eficiente y escalable.
4. Implementar la API REST del chatbot para el reconocimiento de la intención de los usuarios mediante un algoritmo basado en la similitud de oraciones para facilitar el proceso de comunicación entre los demás componentes del proyecto que incluyen el frontend y el backend del chatbot.

1.3 Alcance

El alcance del componente se divide en tres fases fundamentales y una fase de integración final. Todo esto siguiendo un proceso de desarrollo iterativo e incremental, con el propósito de implementar un chatbot especializado en atender procesos académicos de grado para la Facultad de Ingeniería de Sistemas.

Inicialmente, en la fase de diseño o planteamiento se lleva a cabo un análisis de las herramientas de chatbot disponibles, centrándose en sus arquitecturas, funcionamientos y tecnologías predominantes. Simultáneamente, se definen los requerimientos específicos de la Facultad de Ingeniería de Sistemas para la atención de procesos académicos. Como resultado, se obtiene una lista detallada de herramientas analizadas, destacando sus características relevantes, así como un informe completo de los requerimientos más relevantes para la facultad. Además, se lleva a cabo el diseño de la arquitectura del backend del chatbot, representada mediante diagramas UML que ilustran la estructura y la interacción de los componentes.

Con los requerimientos y el diseño del backend, en la fase de implementación se procede con la construcción de la API REST del chatbot. Inicialmente, se prioriza el reconocimiento básico de intenciones basado en recuperación. A medida que avanza la implementación, se optimiza el backend para permitir una interacción en tiempo real entre los usuarios y el chatbot. Los resultados esperados de esta fase incluyen una API REST funcional capaz de reconocer las intenciones iniciales de los usuarios y un backend eficiente para manejar consultas y respuestas de manera efectiva.

Una vez implementado el backend, durante la fase de pruebas unitarias, evaluación y análisis de resultados, el sistema se verifica y valida, especialmente, en lo que respecta al reconocimiento de intenciones por parte de los usuarios. Los resultados de estas pruebas son analizados y se recopila retroalimentación para identificar áreas de mejora. Simultáneamente, se aplican procesos de refactoring para mejorar la cohesión y reducir el acoplamiento entre los componentes. Los entregables de esta fase incluyen un informe detallado de las pruebas realizadas y los resultados obtenidos, así como un backend refinado y mejorado.

Finalmente, se integra el resultado del componente con los demás elementos del proyecto. Se llevan a cabo pruebas adicionales, esta vez con usuarios reales, para validar la eficacia de la integración. Los resultados esperados incluyen una integración exitosa y validada, respaldada por miembros de la Facultad de Ingeniería de Sistemas.

1.4 Marco teórico

A continuación, se presentan los conceptos y tecnologías utilizas en el desarrollo completo del componente. El objetivo de este capítulo es proporcionar un resumen concreto de los conceptos teóricos que intervienen en la implementación del componente, y las tecnologías específicas que se utilizan.

Chatbots

Los chatbots son herramientas o servicios de conversación que permiten a los usuarios recibir atención de forma inmediata, especialmente en áreas de consulta o realización de procesos dentro de las organizaciones. Actualmente, el modo de funcionamiento de los chatbots se divide en dos enfoques principales:

- **Chatbots generativos:**

  Son capaces de generar información basándose en el contexto de las conversaciones que mantienen con los usuarios. Para ello, utilizan algoritmos de aprendizaje automático, lo cual les permite comprender la semántica y las intenciones de las oraciones [2].

  Aunque brindan respuestas variadas y atractivas, su desventaja es que requieren grandes conjuntos de datos de entrenamiento para mejorar su capacidad de respuesta frente a todo tipo de temas.

- **Chatbots basados en recuperación:**

Se centran en analizar las intenciones de los usuarios mediante la coincidencia de sus mensajes con frases de activación guardadas previamente en un repositorio o base de conocimiento. Generalmente, utilizan algoritmos simples para evaluar las consultas de los usuarios y posteriormente seleccionar una respuesta apropiada. Además, suelen utilizar interfaces de programación de aplicaciones (API) para acceder a información específica [2]. Así pues, su ventaja radica en la rapidez con la que pueden encontrar una respuesta sin la necesidad de pasar por un proceso de entrenamiento extenso.

Modelos de chatbots

La mejora continua en el ámbito de los modelos de chatbot ha permitido desarrollar diversos enfoques tecnológicos que se especializan en contextos específicos. Así pues, hoy en día principalmente se consideran dos enfoques:

- **Modelos basados en Transformers:**

A la fecha actual, uno de los enfoques más innovadores son los modelos de chatbots basados en redes neuronales de tipo Transformer. Estas redes utilizan un componente de autoatención para mejorar la eficiencia y comprensión contextual de las conversaciones [3]. Así pues, mediante la autoatención los chatbots son capaces de registrar el contexto completo de las conversaciones para ofrecer respuestas más adecuadas en cualquier tipo de entorno.

- **Modelos basados en redes convolucionales:**

Un modelo de chatbot basado en redes convolucionales utiliza una abstracción de las neuronas humanas para identificar partes del texto y extraer sus características relevantes. Este método ofrece una perspectiva valiosa en la creación de chatbots, especialmente en situaciones donde la interpretación de la información es el aspecto más relevante [4]. Sin embargo, no son muy recomendadas en entornos en los que se cuentan con respuestas predefinidas y solamente se requiere identificar las intenciones de los usuarios.

Similitud textual semántica

Comúnmente, dos piezas de texto pueden compartir significado o contenido semántico, a pesar de tener diferencias notables en su estructura o términos utilizados. Así pues, la similitud textual semántica implica evaluar que tan cercanos están dos fragmentos de texto en términos de su significado o interpretación [5].

Los métodos para medir la similitud textual semántica a menudo se basan técnicas de procesamiento de lenguaje natural y modelos entrenados específicamente para representaciones semánticas, como es el caso de los modelos basados en Transformers. Estos modelos representan las palabras u oraciones en espacios vectoriales conocidos como embeddings, los cuales permite aplicar algoritmos como la similitud del coseno para conocer el grado de similitud entre dos frases.

Procesamiento del lenguaje natural

El procesamiento del lenguaje natural (PLN) es una disciplina clave en la inteligencia artificial que se enfoca en la interacción entre las computadoras y el lenguaje humano. En el contexto del PLN, se llevan a cabo diversas técnicas para analizar y comprender el texto de manera automatizada. Entre las etapas fundamentales se encuentran:

- **Limpieza de sintaxis:**

  La limpieza de sintaxis permite corregir la estructura del texto para que este posteriormente pueda ser utilizado en procesos del reconocimiento de la intención. Para ello, generalmente se aplica la normalización de mayúsculas y minúsculas, y la eliminación de caracteres irrelevantes. En conjunto, todas estas técnicas contribuyen a la coherencia y comprensión del texto.

- **Eliminación de palabras vacías:**

  Las palabras vacías también conocidas como “stop-words” son palabras comunes que no aportan significado relevante en un idioma determinado. Comúnmente, las palabras vacías incluyen artículos, pronombres y preposiciones [6]. Así pues, su eliminación permite focalizar el procesamiento de términos más relevantes, mejorando la precisión de los modelos al reconocer las intenciones de los usuarios.

- **Lematización:**

  Al lematizar una palabra se aplica una técnica algorítmica que permite encontrar la estructura morfológica básica del término en cuestión. Este proceso agrupa palabras de diferentes formas morfológicas, como verbos conjugados y sustantivos con diferente género para tratarlos como un solo elemento que cuenta con el mismo significado.

Marco de trabajo Cynefin

El marco de trabajo Cynefin es un concepto utilizado en diversos campos para facilitar la toma de decisiones y la resolución de problemas. Este marco consta de cinco dominios:

simple, complicado, complejo, caótico y desorden. Su objetivo es comprender la naturaleza de un problema o situación y elegir el enfoque adecuado para abordarlo [7]. Lo cual, suele permitir seleccionar un marco de trabajo adecuado para su gestión. A continuación, se presenta un resumen de los dominios existentes:

- En el dominio simple, la relación causa-efecto es clara y predecible, abordándose mediante buenas prácticas y conocimiento experto.
- En el dominio complicado, la relación requiere análisis y experiencia, resolviéndose mediante la aplicación de buenas prácticas y expertos en la materia.
- En el dominio complejo, la relación solo se percibe retrospectivamente, y las soluciones emergen a través de la exploración y la percepción.
- En el dominio caótico, no hay una relación clara entre causa y efecto, requiriendo acción inmediata para establecer el orden y luego transitar a otro dominio.
- El estado de desorden representa la confusión y la incertidumbre, caracterizado por no saber a cuál de los otros dominios se aplica.

Marco de trabajo Kanban

Kanban es un marco de trabajo que permite gestionar proyectos para mejorar la eficiencia y productividad del flujo de trabajo. Actualmente, se utiliza en diversos campos, sin embargo, ha cobrado gran relevancia en la gestión de proyectos de software debido a su gran flexibilidad y alto nivel de personalización.

Kanban se basa en los principios de manufactura Lean, en los cuales destacan la mejora continua y la satisfacción del cliente [8]. Así pues, al ser un marco flexible puede personalizarse para adaptarse a las necesidades de diferentes equipos.

Modelado y notación de procesos de negocios

BPMN, siglas en inglés de Business Process Model and Notation, es una notación estandarizada para el modelado de procesos de negocios. Utiliza símbolos gráficos para representar de manera clara los pasos en un proceso y las interacciones entre los participantes [9].

En el ámbito del software, BPMN es una herramienta valiosa para especificar requisitos, ya que facilita la comprensión, análisis y mejora de los procesos que existen en una organización involucrada en el desarrollo de un producto de software.

Diagramas UML

En el contexto del software, los diagramas UML (Unified Modeling Language) permiten representar la arquitectura o estructura del código desde diferentes perspectivas para facilitar su comprensión. Existen varios tipos de diagramas UML, sin embargo, dos de los más utilizados son los siguientes:

- **Diagramas de paquetes:**

  Permiten observar las dependencias entre diferentes componentes de un mismo proyecto. Así pues, los diagramas de paquetes son adecuados para representar la estructura de un proyecto cuando otros tipos de diagramas sean demasiado complejos o cuando se está planificando la estructura general de los sistemas [10].

- **Diagramas de clases:**

  Son útiles para representar la estructura o arquitectura del sistema de forma estática, por lo cual, son muy utilizados para modelar sistemas orientados a objetos. Los diagramas de clase permiten entender de manera efectiva la arquitectura y la composición del sistema, facilitando la comunicación y proporcionando una base sólida para el desarrollo y la implementación del software [10].

Patrones de diseño de software

Contenido.

- **Chain of Responsability:**

  Permite que una solicitud pase a través de una cadena de manipuladores, los cuales pueden procesarla o modificarla. Cada manipulador decide si procesa la solicitud o si la pasa al siguiente elemento de la cadena [11]. De esta forma, se pueden procesar estructuras de datos de forma secuencial.

  Por ejemplo, en el contexto de un chatbot, el patrón Chain of Responsability es adecuado para aplicar técnicas de procesamiento de lenguaje natural, tales como limpieza de sintaxis, eliminación de palabras vacías y lematización.

- **Factory Method:**

  Permite definir una interfaz para crear objetos de forma dinámica mediante la delegación de la decisión de su tipo concreto a las subclases. Esto brinda la posibilidad de que una clase delegue la responsabilidad de instanciación a sus subclases [11].

En el contexto del desarrollo de un chatbot, este patrón resulta particularmente útil para incorporar nuevas tecnologías encargadas del procesamiento del lenguaje natural o el reconocimiento de las intenciones de los usuarios.

- **Proxy:**

Permite crear un representante de otro objeto con el fin de controlar el acceso a este. Este patrón es útil cuando se desea agregar una capa de control o seguridad adicional antes de acceder a un objeto en específico [11]. Así pues, se lo utiliza para el desarrollo de sistemas que requieren de una capa extra de seguridad.

- **Singleton:**

Garantiza la existencia de una única instancia de una clase y proporciona un punto de acceso global a ella [11]. El patrón Singleton es particularmente útil cuando se debe compartir una única instancia en toda la aplicación.

En el contexto del uso de modelos para el procesamiento de lenguaje natural, el patrón Singleton permite que varias consultas utilicen un mismo modelo para evitar consumos excesivos de memoria en ambientes con pocos recursos de hardware.

Pruebas de software

En el contexto del desarrollo de software, las pruebas son una técnica que permite evaluar la calidad de los sistemas para garantizar el cumplimiento de los requisitos establecidos. A continuación, se detallan dos de los tipos de pruebas más utilizados:

- **Pruebas unitarias:**

Estas pruebas se centran en evaluar pequeñas unidades individuales de código para comprobar si su funcionamiento es correcto. Específicamente, este tipo de pruebas permiten detectar y corregir posibles errores a nivel de código [12].

Además, son adecuadas para validar la entrega de pequeños incrementos o modificaciones simples que se realizan durante procesos de mejora continua.

- **Pruebas de carga:**

Permiten evaluar el rendimiento de los sistemas ante diversas cargas de trabajo. Así pues, son útiles para validar si las aplicaciones se comportan de forma adecuada al procesar grandes volúmenes se solicitudes o transacciones.

Python

Python es un lenguaje de programación extremadamente versátil debido a su gran cantidad de frameworks y librerías. Es particularmente útil para implementar modelos neuronales y para desarrollar programas basados en inteligencia artificial. Además, se integra perfectamente con un gran número de herramientas que amplían su uso a entornos des escritorio, móviles o web.

- **PEP 8:**

  PEP 8 es un estándar de Python creado para garantizar la coherencia y claridad del código fuente. Su uso permite que los miembros de un equipo colaboren en conjunto bajo las mismas reglas para reducir la posibilidad de errores [13]. Así pues, al utilizar PEP 8 se promueve el desarrollo eficiente y sostenible de aplicaciones.

- **Flask:**

  Flask es un framework de desarrollo web para Python que permite crear aplicaciones web de forma sencilla y con gran flexibilidad. Su funcionamiento se enfoca en un diseño minimalista y una estructura modular completamente personalizable [14]. De este modo, Flask es una opción adecuada para desarrollar proyectos web de cualquier tamaño de forma simple.

- **Scikit-Learn:**

  Scikit-Learn es una extensa biblioteca de aprendizaje automático para Python. Proporciona herramientas eficientes y fáciles de utilizar para diversos entornos de aprendizaje automático, incluyendo el cálculo de semejanza mediante la similitud del coseno [15]. Su funcionamiento se destaca por su rápida y fácil integración con el lenguaje de programación, lo cual lo hace adecuado para elaborar sistemas basados en inteligencia artificial.

2 METODOLOGÍA

La metodología adoptada para el desarrollo del chatbot se estructura en varias fases, cada una crucial para garantizar un proceso coherente y efectivo. La primera fase consiste en un "Análisis previo de requisitos", en el que se evalúan las necesidades del proyecto utilizando el marco Cynefin, clasificando el dominio como complicado y seleccionando el marco Kanban para la gestión.

En la "Especificación de requisitos", se detallan los requisitos específicos del backend del chatbot, destacando la complejidad asociada a la identificación precisa de soluciones en un contexto único.

Luego, la "Fase de diseño o planteamiento" se centra en proponer un diseño eficiente, considerando elementos como el uso de un proxy para el control de acceso y componentes específicos para tareas cruciales como el reconocimiento de la intención de los usuarios.

La "Fase de implementación" se gestiona mediante el marco de trabajo Kanban y se enfoca en la programación del motor de identificación de la intención, destacando la elección de Python y Flask para la implementación.

Así también, en la "Fase de pruebas, evaluación y análisis de resultados" se lleva a cabo una prueba manual y se exploran modelos neuronales para mejorar la precisión, culminando con la integración exitosa del backend. Esta metodología integral garantiza un desarrollo coherente y optimizado del chatbot, alineado con los objetivos y requisitos específicos de la Facultad de Ingeniería de Sistemas.

Por último, en la “Fase de integración” se realizan pruebas de carga para validar que la configuración de hardware es apta para el adecuado funcionamiento del chatbot. Así también, el chatbot se integra con los componentes de front-end y persistencia de datos. De tal manera, el componente descrito presenta un funcionamiento completo del sistema del chatbot.

2.1 Análisis previo de requisitos

El análisis previo de requisitos se utilizó como una fase fundamental que se realiza previo al diseño y desarrollo del componente para definir y comprender las necesidades específicas que deben considerarse durante la construcción del backend del chatbot destinado a la atención de procesos académicos de grado en la Facultad de Ingeniería de Sistemas (FIS). Esta etapa se centra en definir las necesidades reales de los involucrados

en el proyecto mediante la realización de reuniones informales con el personal administrativo del subdecanato responsable de gestionar y brindar atención sobre los procesos académicos de grado.

Evaluación inicial de requisitos

Durante una primera reunión, se enfatiza la necesidad de implementar un chatbot capaz de satisfacer las inquietudes de los estudiantes en relación con los procesos académicos de grado. En este contexto, se rescata el requisito a nivel de negocio mostrado en la Tabla 1, el cual sirve como pilar fundamental para la recopilación de requisitos y el posterior desarrollo del sistema.

Tabla 1. Requisito a nivel de negocio del subdecanato de la FIS.

Requisito a nivel de negocio
Como subdecanato de la FIS, quiero mejorar la atención brindada a los estudiantes con respecto a las consultas sobre los procesos académicos de grado para elevar el nivel de satisfacción de los estudiantes.

El requisito se asocia con la creación de un mecanismo eficiente mediante el cual los estudiantes puedan plantear preguntas relacionadas con los procesos académicos de grado, y estas, a su vez, sean respondidas de forma clara y pertinente. Para lograr este objetivo, se establece que el mecanismo utilizado será un chatbot que contará con un conjunto de preguntas frecuentes predefinidas, las cuales serán asociadas a respuestas elaboradas de antemano.

Restricciones del entorno

En el proceso de definición de requisitos para el desarrollo del chatbot destinado a la atención de preguntas frecuentes de los estudiantes de la Facultad de Ingeniería de Sistemas, se identifican restricciones cruciales que deben considerarse para garantizar la viabilidad de la solución propuesta. Estas restricciones, establecidas a partir de la retroalimentación proporcionada durante una reunión con la subdecana de la facultad, recalcan la necesidad de adaptabilidad y sostenibilidad de la herramienta.

- **Gratuidad de la solución:**

  La solución elegida para el desarrollo del chatbot debe ser completamente gratuita. Esta decisión se fundamenta en la intención de evitar costos adicionales para la facultad y garantizar que la implementación del chatbot no genera cargos de

mantenimiento. A pesar de existir soluciones en línea similares que ofrecen servicios parecidos, muchas de ellas están asociadas con suscripciones pagadas. Por lo cual, la gratuidad de la solución es esencial para asegurar la sostenibilidad del proyecto a largo plazo.

- **Escalabilidad y personalización completa:**

  La solución seleccionada debe ser capaz de escalarse fácilmente para adaptarse a las necesidades futuras de la FIS y permitir una personalización completa. Aunque existen diversas alternativas en el mercado, la subdecana subraya la importancia de contar con un chatbot que pueda ajustarse a los cambios que pueda requerir la facultad en el futuro. Así pues, la escalabilidad es esencial para garantizar la eficiencia continua del sistema, mientras que la capacidad de personalización completa permitirá adaptar el chatbot a las necesidades específicas de la FIS.

Estas restricciones del entorno son tomadas como guía para la elección de las tecnologías utilizadas y la plataforma para el desarrollo y mantenimiento del chatbot, asegurando que la solución final sea funcional, efectiva, sostenible y adaptable.

Selección del marco de trabajo

En el proceso de determinar los requisitos es necesario definir el marco de trabajo más adecuado para el desarrollo del backend del chatbot. Para ello, se realiza una evaluación del contexto del componente utilizando el marco Cynefin. Asimismo, se selecciona el marco de trabajo Kanban para la gestión y desarrollo, tomando en cuenta las características particulares del equipo y las necesidades específicas del componente.

Dominio del componente:

De acuerdo con el marco Cynefin, el contexto del componente para desarrollar el backend del chatbot se sitúa en el dominio complicado. Esta clasificación se justifica por la naturaleza del requisito identificado. El cual, muestra una relación causa y efecto casi evidente, lo que implica que, aunque no necesariamente puede no ser simple, existe una comprensión clara de cómo se pueden abordar las consultas de los estudiantes.

Además, es evidente que existen varios tipos de soluciones, sin embargo, la complejidad radica en determinar cuál de esas soluciones es la más adecuada para la facultad considerando las restricciones identificadas. Dada la ausencia de proyectos similares en la institución, el diseño y la planificación son aspectos cruciales para garantizar el éxito del desarrollo del backend del chatbot.

Marco de trabajo Kanban:

El marco de trabajo seleccionado para la gestión y ejecución del proyecto es Kanban. Esta elección se fundamenta en varios aspectos específicos del componente y del equipo encargado de su desarrollo:

- **Dominio del problema:**

  Principalmente, la elección de Kanban se basa en la clasificación del dominio complicado del componente según el marco de trabajo Cynefin. En este contexto, donde existe una relación clara de causa y efecto, pero se enfrenta la complejidad de determinar la solución más adecuada entre varias opciones, Kanban se presenta como una elección apropiada. Esto debido a que Kanban permite adaptarse a la naturaleza del problema, facilitando la gestión de múltiples soluciones potenciales y ajustándose a las restricciones identificadas.

- **Tamaño del equipo:**

  Dado que el equipo encargado del desarrollo del backend se reduce a una sola persona, se requiere un enfoque ágil y flexible que se adapte a la dinámica de trabajo de un solo desarrollador sin sobrecargarlo.

- **Enfoque en requerimientos individuales:**

  La flexibilidad de Kanban permite centrarse en la gestión individual de requerimientos, lo cual será crucial para mantener el enfoque del desarrollador en las tareas más relevantes en un determinado periodo de tiempo.

La elección del tablero Kanban para la gestión del proyecto se basa en la necesidad de establecer un marco de trabajo simple que mantenga un alto nivel de flexibilidad y adaptabilidad de acuerdo con las características del equipo y el componente. La Figura 1 detalla la estructura específica del tablero Kanban propuesto.

Figura 1. Estructura del tablero Kanban propuesto para el desarrollo del componente.

A continuación, se describe la estructura propuesta incluyendo las características que se utilizarán con las tarjetas Kanban:

- **Columnas para la especificación, desarrollo, pruebas e integración:**
    - **Especificación:** La columna de Especificación desempeñará un papel crucial al inicio del ciclo de vida del proyecto. Esta fase se concentrará en definir y clarificar los requisitos identificados antes de que ingresen a la fase de desarrollo. Aquí, se llevará a cabo un proceso de análisis detallado en colaboración con las partes interesadas, lo que permitirá establecer una comprensión clara y unificada de los objetivos del proyecto. Las tarjetas en esta columna contendrán detalles esenciales sobre los requisitos, incluidas las consultas y discusiones que surjan durante el proceso de especificación.
    - **Desarrollo:** La columna de Desarrollo es el núcleo del tablero Kanban, donde se realizará un seguimiento constante del progreso de las tareas en desarrollo. Esta sección proporcionará una visualización clara de las actividades en curso y las que ya han alcanzado la finalización.
    - **Pruebas:** La columna de Pruebas se centrará en la verificación y validación de las funcionalidades implementadas en la fase de desarrollo. Aquí, se llevarán a cabo actividades de prueba para garantizar que cada componente del chatbot cumpla con los requisitos especificados y funcione de acuerdo con las expectativas.
    - **Integración:** La columna de Integración desempeñará un papel esencial al final del ciclo de desarrollo. Su objetivo principal será garantizar la integración fluida de las distintas partes del sistema. Esta fase es crítica para identificar posibles conflictos y asegurar que todas las partes del sistema trabajen en conjunto.
- **Etiquetas para prioridad y clasificación:**
    - **Etiquetas para prioridad:** La diferenciación de prioridades Normal y Urgente mediante etiquetas proporcionará una indicación visual instantánea del nivel de importancia y urgencia de cada tarea. Esto facilitará la toma de decisiones rápida y la asignación efectiva de recursos.
    - **Etiquetas para componentes:** La inclusión de etiquetas asociadas a los requisitos funcionales y no funcionales en las tarjetas Kanban mantendrá la trazabilidad de los componentes desarrollados en cada tarea. Lo cual permitirá enfocar los esfuerzos del desarrollador.

- **Backlog para trazabilidad:**

  El uso de un backlog para almacenar los requisitos a nivel de negocio y de usuario asegurará la trazabilidad completa desde la concepción hasta la implementación. Esto proporcionará una vista clara de las tareas pendientes y permitirá priorizar su desarrollo de manera efectiva.

Cabe mencionar que la estructura de tablero Kanban propuesta tiene el objetivo de ofrecer una herramienta visual y flexible que permita segmentar las fases del proyecto y organizar el flujo de trabajo del desarrollador. Además, busca facilitar la identificación rápida de tareas, promoviendo una gestión eficiente y transparente del desarrollo.

Levantamiento formal del proceso

El proceso de atención de procesos académicos de grado de la Facultad de Ingeniería de Sistemas (FIS) es formalmente establecido a través de reuniones y un proceso de observación dirigido al personal administrativo del subdecanato. La representación formal de este proceso se consigue mediante el uso de la notación de Modelado de Procesos de Negocio (BPMN), obteniendo el resultado mostrado en la Figura 2.

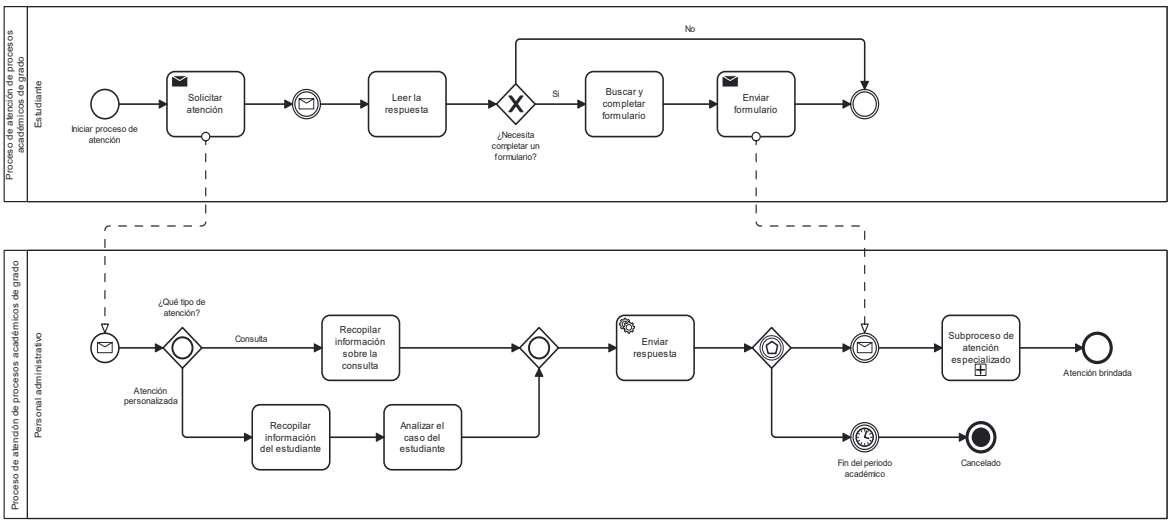

Figura 2. Diagrama del proceso de atención de procesos académicos de grado.

La observación detallada del proceso revela la existencia de dos componentes principales que plantean desafíos significativos y que el desarrollo del chatbot debe abordar de manera eficiente para garantizar la cobertura de las expectativas de los involucrados:

1. **Tiempo de espera entre consulta y respuesta:**

   El análisis del proceso permite identificar que existe un tiempo de espera relevante entre el momento en que los estudiantes realizan una consulta de forma telemática y el instante en que el personal administrativo puede proporcionar una respuesta. Este retraso puede generar una experiencia de usuario insatisfactoria y afectar la eficiencia general del proceso de atención.

2. **Dificultad e incertidumbre en la búsqueda de formularios o documentos:**

   Otro aspecto crítico que se detecta es la dificultad e incertidumbre que enfrentan los estudiantes al buscar formularios o documentos necesarios para avanzar en el proceso de atención. Esta complicación puede generar pérdida de tiempo y aumentar la probabilidad de errores en el cumplimiento de los requisitos académicos.

La representación visual del proceso a través del diagrama BPMN proporciona una comprensión clara de los puntos de fricción en el flujo de trabajo actual. Estos hallazgos muestran la necesidad de que el chatbot, como mecanismo de solución, cuente con características específicas para optimizar el proceso de atención.

En particular, el chatbot deberá ofrecer respuestas rápidas y precisas para reducir el tiempo de espera y facilitar la búsqueda y recuperación de información relevante para los estudiantes.

2.2 Especificación de requisitos

Durante las reuniones informales y el levantamiento formal del proceso, se logran identificar dos requisitos funcionales fundamentales que se encuentran asociados directamente con el requisito a nivel de negocio que se mostró en la Tabla 1. Adicionalmente, en el transcurso de las conversaciones y negociaciones surge la necesidad de mantener la seguridad de los servicios tecnológicos de la FIS, dando lugar a la identificación de un requisito no funcional de gran importancia.

Los requisitos identificados se muestran en la Tabla 2, en la cual se incluye el tipo de requisito al que pertenecen y el identificador que permitirá mantener la trazabilidad durante la etapa de desarrollo.

Tabla 2. Requisitos funcionales y no funcionales identificados.

Tipo de requisito	Identificador	Descripción
Requisito funcional	RF-01	Como subdecanato de la FIS, quiero ofrecer respuestas rápidas y coherentes a las consultas de los estudiantes, para optimizar la asignación de recursos y agilizar la atención de procesos académicos de grado.
	RF-02	Como subdecanato de la FIS, quiero que los estudiantes encuentren fácilmente los formularios o documentos necesarios, para mejorar su experiencia y optimizar el proceso de atención de procesos académicos de grado.
Requisito no funcional	RNF-01	Como subdecanato de la FIS, quiero mantener la seguridad de los servicios tecnológicos asociados a la atención de procesos académicos de grado, para proteger a los estudiantes de posibles amenazas cibernéticas.

Así también, la Figura 3 muestra el estado del tablero Kanban tras el análisis previo de requisitos. En este punto, las actividades y tareas incluidas en el tablero se enfocan en el desarrollo completo de la fase de diseño o planteamiento.

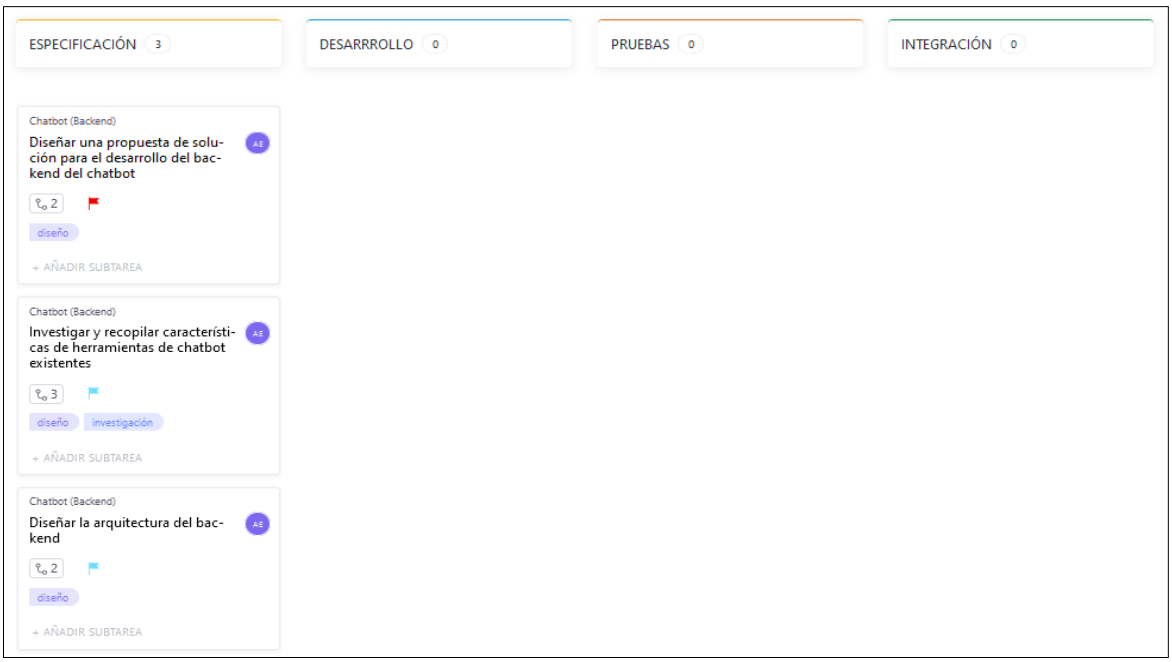

Figura 3. Actividades y tareas determinadas para la fase de diseño o planteamiento.

2.3 Fase de diseño o planteamiento

Durante esta fase el objetivo principal es proponer una solución y diseñar su arquitectura mediante un análisis de herramientas de chatbot. Este análisis permite identificar las características esenciales que adoptan las actuales tecnologías de chatbot para ofrecer respuestas ante problemáticas similares.

Así pues, al explorar las capacidades, tecnologías y arquitecturas predominantes, es posible establecer las bases para implementar el núcleo del chatbot en cuanto al reconocimiento inicial de intenciones mediante recuperación. Este enfoque proporciona una herramienta sólida para el desarrollo, permitiendo una comprensión clara de las interacciones iniciales del chatbot con los usuarios.

Propuesta y validación de la solución

Considerando los requisitos funcionales y no funcionales previamente identificados, se propone una solución integral que aborda eficazmente las necesidades identificadas. Para ello, se considera la implementación de un proxy que se utilizará como un componente clave para controlar el acceso a la lógica del negocio del chatbot, filtrando consultas y previniendo la ejecución de scripts o sentencias SQL no autorizadas. Además, la inclusión de un componente dedicado a las operaciones CRUD optimizará la escalabilidad del sistema, ofreciendo una interfaz amigable para la modificación de frases de activación y respuestas del chatbot.

Así también, para garantizar una mayor flexibilidad y reducir el acoplamiento del sistema, se proponen dos componentes especializados. Un módulo de filtrado, encargado de tareas como la eliminación de símbolos y palabras irrelevantes, se aplicará tanto a las frases de los usuarios como a los documentos de la base de conocimiento. Asimismo, se incorporará un componente exclusivo para el reconocimiento de las intenciones de los usuarios, permitiendo una adaptabilidad eficiente a través de la incorporación de nuevos algoritmos según sea necesario.

En la Figura 4 se adjunta el diagrama de paquetes que ilustra la solución propuesta, destacando la interconexión y funcionalidad de cada módulo mencionado. Se recalca la necesidad de reducir el acoplamiento del sistema haciendo que cada módulo trabaje independientemente de los demás para promover la escalabilidad del programa.

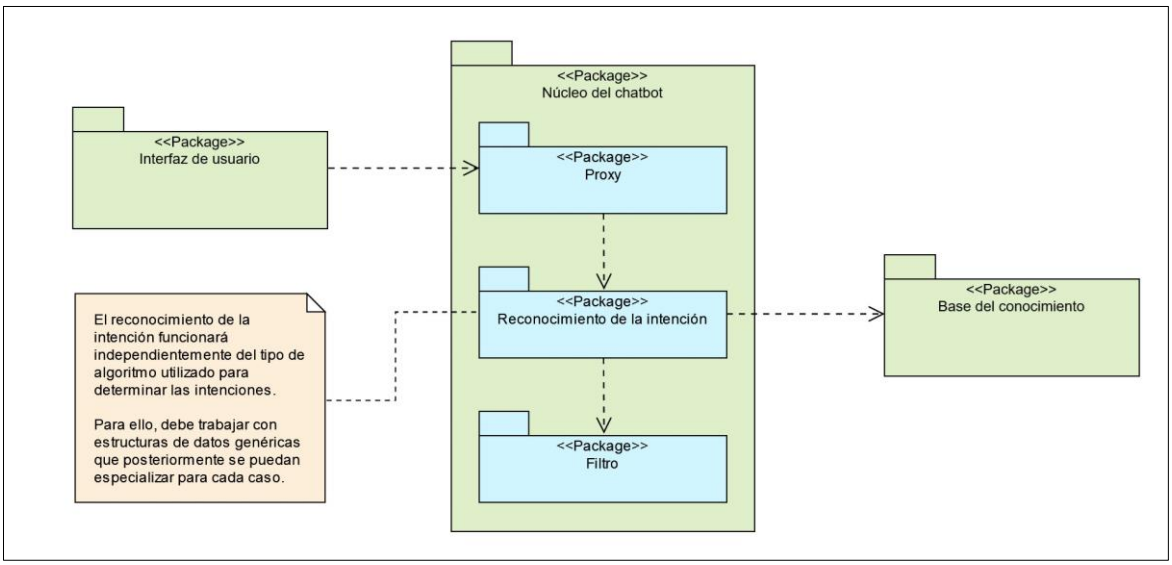

Figura 4. Diagrama de paquetes de la solución propuesta para el backend del chatbot.

La solución propuesta es validada en colaboración con todos los miembros del proyecto, asegurando un consenso respecto al esquema de comunicación entre los diversos componentes. Esta validación garantiza una comprensión compartida y coherente entre el equipo, fortaleciendo la eficacia en la ejecución del proyecto.

Investigación y recopilación de características

Durante la recopilación de características de relevancia se realiza un trabajo de campo en el cual se definen los criterios de evaluación fundamentales que orientarán el desarrollo del chatbot. A través de reuniones con la subdecana de la FIS y un análisis detallado de la arquitectura tecnológica de la facultad, se establece un grupo de criterios de evaluación meticulosamente elaborados. Cada criterio se desglosa en características específicas, priorizadas según las necesidades y preferencias de la FIS.

El Anexo II proporciona un listado completo de estos criterios de evaluación, detallando sus respectivas consideraciones, características, justificaciones y puntajes asignados. Esta información será crucial para identificar las características más relevantes que serán implementadas en el backend del chatbot. Además, ofrecerá un conjunto de características escalables, permitiendo la adaptación del sistema a futuras necesidades. En la Tabla 3 se destacan los criterios de evaluación clave con el objetivo de brindar una visión general de los aspectos prioritarios de la FIS para el desarrollo del chatbot.

Tabla 3. Listado de criterios de evaluación para el análisis de herramientas de chatbot.

Criterios de evaluación de las herramientas de chatbot
Compatibilidad del servicio del chatbot con la versión de Joomla utilizada por la Facultad de Sistemas (Joomla 3.8.2).
Precio de acceso al servicio del chatbot.
Número de consultas permitidas por el servicio del chatbot.
Plataformas de despliegue ofrecidas por el servicio del chatbot.
Grado de completitud de las actividades del proceso de consulta e información estudiantil que el servicio del chatbot es capaz de realizar.
Nivel de personalización de la interfaz ofrecida por el servicio del chatbot.
Grado de complejidad de configuración y entrenamiento del servicio del chatbot.
Existencia de documentación y soporte técnico para el servicio del chatbot.
Mantenibilidad del servicio del chatbot.
Valoración del evaluador tras utilizar el servicio del chatbot.

Análisis de herramientas de chatbot

En el contexto del análisis de servicios de chatbot, se realiza una búsqueda de soluciones destacadas en la industria, resultando en la identificación de 11 herramientas potenciales para la consideración de implementación en la Facultad de Ingeniería de Sistemas. Este proceso se basa en la recopilación de recomendaciones y en la identificación de herramientas ampliamente utilizadas en el ámbito. Las herramientas seleccionadas abarcan una variedad de proveedores y enfoques tecnológicos, proporcionando así una gama diversa de opciones para la evaluación.

Los resultados de estas evaluaciones se detallan en el Anexo III, donde se presentan las puntuaciones asignadas a cada servicio de chatbot y las observaciones extraídas en relación con las tecnologías utilizadas, las arquitecturas predominantes y las restricciones asociadas a su implementación. Este análisis permite una evaluación integral de las herramientas, proporcionando una visión detallada de sus fortalezas y debilidades en relación con los criterios de evaluación previamente establecidos.

Un resumen de los resultados conseguidos tras la evaluación se presenta en la Tabla 4, en la cual se detallan las puntuaciones finales de cada servicio evaluado para entornos de baja (promedio de 1 consulta diaria por estudiante) y alta (promedio de 3 consultas diarias por estudiante) demanda del servicio (Anexo I).

Tabla 4. Resultados de la evaluación de los servicios de chatbot seleccionados.

Servicio de chatbot	Puntuación (Sobre 100)	
	Baja demanda	Alta demanda
Kore.ai	80.9	80.4
ChatGPT	71.0	72.2
ChatBot	74.3	68.2
Freshchat	69.5	69.0
ItsAlive	43.7	43.7
Bold360	57.8	58.5
Reply.ai	48.3	49.3
Botsify	64.9	65.8
Flow XO	68.9	70.0
Acquire.io	51.8	53.0

Los hallazgos de la evaluación sugieren que, en el momento actual, la implementación de alguno de los servicios de chatbot analizados no es factible para la FIS. Las restricciones asociadas a los límites de consulta y los costos vinculados a estos servicios son catalogados como impedimentos significativos.

Como conclusión, se enfatiza la necesidad de optar por un desarrollo personalizado del backend del chatbot. Esta decisión se fundamenta en la búsqueda de una solución personalizada que garantice la cobertura de características esenciales y ofrezca una escalabilidad robusta para futuras necesidades.

En consecuencia, la Tabla 5 presenta las características seleccionadas para el desarrollo del chatbot tras un proceso de negociación. Estas características específicas se identificaron como las más relevantes para satisfacer los requisitos funcionales.

Tabla 5. Características seleccionadas para el desarrollo del backend del chatbot.

Criterio de evaluación	Características seleccionadas
Compatibilidad del servicio del chatbot con la versión de Joomla utilizada por la Facultad de Sistemas (Joomla 3.8.2).	Conexión con el servicio del chatbot mediante código JavaScript (e.g. conexiones a través del protocolo WebSocket o servicios web SOAP o REST).

Precio de acceso al servicio del chatbot.	La solución desarrollada será completamente gratuita y no impondrá ningún costo asociado a su mantenimiento.
Número de consultas permitidas por el servicio del chatbot.	La solución desarrollada no tendrá ningún límite de consultas permitidas para los estudiantes.
Plataformas de despliegue ofrecidas por el servicio del chatbot.	Despliegue del servicio del chatbot en el sitio web de la FIS.
Grado de completitud de las actividades del proceso de consulta e información estudiantil que el servicio del chatbot es capaz de realizar.	El chatbot será capaz de ofrecer respuestas ante las preguntas más frecuentes de los estudiantes. Incluyendo la presentación de recursos (e.g. enlaces, imágenes, videos, etc.).
Nivel de personalización de la interfaz ofrecida por el servicio del chatbot.	Personalización de las interfaces del chatbot mediante HTML, CSS o JavaScript directamente desde su plataforma de configuración (e.g. modificar los estilos CSS directamente desde la plataforma de creación del chatbot).
Grado de complejidad de configuración y entrenamiento del servicio del chatbot.	Entrenamiento manual del chatbot mediante la creación de modelos u otros conceptos relacionados con Machine Learning.
Existencia de documentación y soporte técnico para el servicio del chatbot.	La solución desarrollada contará con diagramas UML que ofrecerán un enfoque general sobre la arquitectura del chatbot.
Mantenibilidad del servicio del chatbot.	La solución desarrollada podrá ser gestionada directamente por la FIS.

Establecimiento del algoritmo para la identificación de la intención

En el proceso de establecer el algoritmo para la identificación de la intención en el backend del chatbot, inicialmente se optará por un enfoque basado en la similitud del coseno. Esta elección se fundamenta en la necesidad de responder eficientemente a preguntas frecuentes, una característica clave seleccionada para el chatbot. Así pues, el algoritmo de la similitud del coseno se considera una opción adecuada debido a su sencilla implementación y a la naturaleza específica de la base de conocimiento que se construirá en otros componentes del proyecto.

En este sentido, la similitud del coseno será especialmente adecuada considerando una base de conocimiento compuesta por un conjunto predefinido de preguntas frecuentes vinculadas a respuestas específicas. Lo cual permitirá obtener una de las respuestas asociadas a un determinado conjunto de preguntas frecuentes.

La selección de la similitud del coseno implicará comparar los mensajes de los estudiantes con las preguntas frecuentes almacenadas, permitiendo determinar a cuál de ellas se refiere el mensaje. Este enfoque proporciona una respuesta rápida y adecuada para un reconocimiento de intenciones en un tiempo relativamente breve. No obstante, también se contempla la posibilidad de mejorar la precisión del chatbot mediante la incorporación de modelos neuronales que realicen un proceso similar.

Diseño de la arquitectura

A partir de las especificaciones realizadas hasta el momento, se propone una estructura robusta para el backend del chatbot, cubriendo tanto la identificación de intenciones de los estudiantes como la obtención de respuestas a mensajes específicos. En la Figura 5, se presenta el diagrama de clases UML para el reconocimiento de intenciones basado en la similitud del coseno, destacando la implementación de tres patrones de diseño clave.

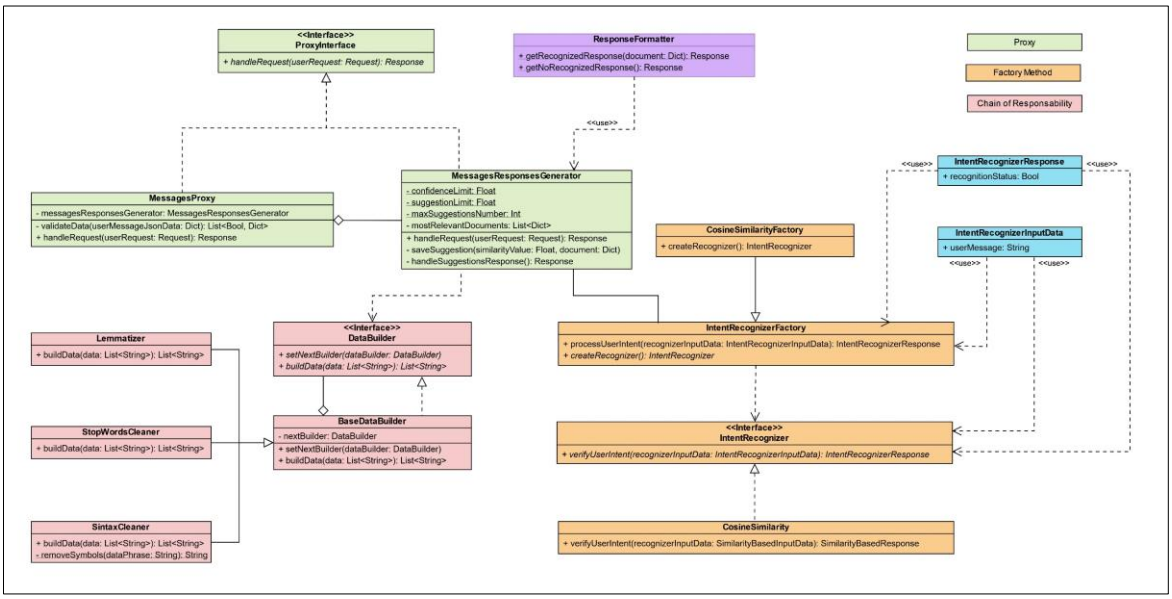

Figura 5. Diagrama de clases para el módulo del reconocimiento de la intención.

En primer lugar, se emplea el patrón de diseño Proxy como capa expuesta de la API REST, encargada de validar y controlar las peticiones dirigidas al backend. Este enfoque fortalece la seguridad y garantiza un manejo eficiente de las solicitudes, evitando accesos no

autorizados y asegurando un flujo controlado de información. El patrón Chain of Responsibility se utiliza para depurar y filtrar la información antes de que sea analizada por el componente de reconocimiento de intenciones. Este patrón facilita la eliminación de símbolos y palabras no deseadas, transformándolas en lemas y optimizando así el proceso de identificación de intenciones.

Además, se incorpora el patrón de diseño Factory Method para construir el motor de reconocimiento de intenciones. Esta elección permite una gran adaptabilidad a través de la incorporación de nuevos algoritmos o modelos neuronales sin alterar la estructura del backend. La flexibilidad de Factory Method optimizará la implementación de mejoras futuras, garantizando una evolución continua y eficiente del sistema.

En la Figura 6, se presenta el diagrama de clases UML para el diseño de la arquitectura del chatbot centrado en la obtención de respuestas a mensajes específicos. Nuevamente, se destaca el uso del patrón Proxy como capa de protección en la API REST. Este patrón asegura la integridad y seguridad del sistema, actuando como un intermediario que controla y valida las solicitudes dirigidas al backend.

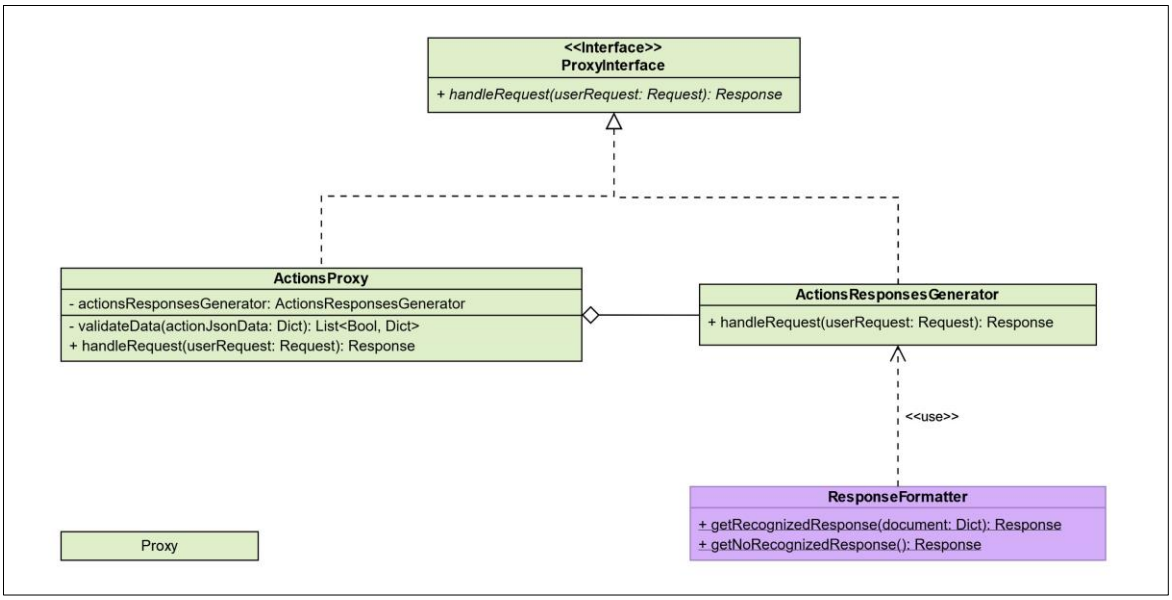

Figura 6. Diagrama de clases para el módulo de obtención de respuestas específicas.

En ambas propuestas, el objetivo de los patrones de diseño de software es proporcionar una estructura modular y escalable que se adapte eficazmente a las necesidades actuales y futuras del desarrollo del backend del chatbot.

Diseño de las estructuras de datos

En la arquitectura del backend del chatbot, se emplearán estructuras de datos específicas para gestionar la información proveniente de la base de conocimiento. Como lo muestra la Figura 7, cada documento se representará como un diccionario, destacándose dos claves fundamentales. La clave "utterances" contendrá las frases asociadas al documento, constituyendo la base para el proceso de comparación mediante el algoritmo de similitud del coseno. Simultáneamente, la clave "responses" albergará las respuestas proporcionadas por el chatbot si la intención del usuario se relaciona con ese documento. Estas respuestas serán del tipo JSON, pudiendo ser de texto, enlace, imagen o referencias a otros documentos para facilitar recomendaciones o sugerencias.

Para las respuestas ofrecidas por el chatbot al identificar la intención del usuario, se utilizará la estructura de datos "responses" previamente descrita. Este objeto JSON, presentado en la Figura 7, se compone de los tipos anteriormente mencionados para ofrecer la posibilidad de brindar respuestas atractivas, permitiendo una comunicación más efectiva y personalizada con los usuarios.

```
{
    "id": "doc-id",
    "title": "Document Title",
    "utterances": [
        "Utterance 1",
        "Utterance 2",
        "Utterance 3"
    ],
    "responses": [
        {
            "type": "text",
            "content": "Text response."
        },
        {
            "type": "image",
            "path": "http://image-path.png",
            "alt": "Image description"
        },
        {
            "type": "link",
            "path": "https://url.com",
            "name": "Link name"
        },
        {
            "type": "action",
            "action": "action-id",
            "name": "Action name"
        }
    ]
}
```

```
{
    "responses": [
        {
            "type": "text",
            "content": "Text response."
        },
        {
            "type": "image",
            "path": "http://image-path.png",
            "alt": "Image description"
        },
        {
            "type": "link",
            "path": "https://url.com",
            "name": "Link name"
        },
        {
            "type": "action",
            "action": "action-id",
            "name": "Action name"
        }
    ]
}
```

Figura 7. Estructuras de datos de los documentos en la base de conocimiento (izquierda) y de las respuestas brindadas por el chatbot (derecha).

El backend del chatbot recibirá los mensajes de los usuarios por medio de la clave “user_message” mostrada en la estructura de datos de la Figura 8. Finalmente, en situaciones de error el backend responderá con una estructura de datos específica que informará sobre la ocurrencia de un error.

Se devolverá un objeto JSON igual al mostrado en la Figura 8. Este objeto contendrá la clave "error", acompañada de un mensaje descriptivo que facilitará la identificación y resolución de problemas.

```
{
  "user_message": "Plain text user message"
}
```

```
{
  "error": "Error message"
}
```

Figura 8. Estructuras de datos de los mensajes aceptados por el backend del chatbot (izquierda) y de los mensajes de error devueltos (derecha).

En este punto, la Figura 9 indica el estado del tablero Kanban tras el diseño y planteamiento de la solución propuesta. Así pues, el tablero incluye las actividades y tareas que se esperan realizar durante la fase de implementación.

Figura 9. Actividades y tareas determinadas para la fase de implementación.

2.4 Fase de implementación

Durante esta fase, el enfoque se centra en llevar a cabo la implementación completa del diseño propuesto para la solución. Esta etapa representa la culminación de la planificación previa, donde se ejecutaron las actividades de desarrollo delineadas en la fase anterior. El objetivo primordial es transformar las ideas y conceptos concebidos durante la fase de diseño en una solución funcional, por lo cual, se incluyen las partes más relevantes del código desarrollado.

Implementación del motor de identificación de la intención

Para la implementación de la solución propuesta se opta por Python como lenguaje de programación principal, debido a su simplicidad y flexibilidad. Esta elección se alinea con la naturaleza del proyecto y permitirá un desarrollo eficiente cuando se presenten cambios a futuro. Así también, se selecciona Flask como framework de desarrollo debido a su curva de aprendizaje adecuada y su capacidad para integrarse de manera fluida con otros componentes que se puedan incorporar en el sistema.

El módulo de reconocimiento de la intención se implementa siguiendo un enfoque de sanitización de datos, incluyendo los mensajes de los usuarios y las frases asociadas a los documentos de la base de conocimiento. Para ello, se implementa el patrón Chain of Responsability que se incluyó durante la fase de diseño. La implementación de este patrón permite facilitar el proceso de sanitización mediante la creación de los objetos respectivos para procesar los datos, tal como se indica en la Figura 10.

```python
def handle_request(self, user_request: Request) -> Response:
    """
    Manage a message request submitted a users to give an adequate response.

    :param user_request: A message request submitted by a user.
    :return: The response generated by the messages service.
    """

    # Create data construction objects to build data
    syntax_cleaner: DataBuilder = SyntaxCleaner()
    stop_words_cleaner: DataBuilder = StopWordsCleaner()
    lemmatizer: DataBuilder = Lemmatizer()

    syntax_cleaner.set_next_builder(stop_words_cleaner)
    stop_words_cleaner.set_next_builder(lemmatizer)
```

Figura 10. Implementación del patrón Chain of Responsability utilizado para el proceso de sanitización de datos.

Posteriormente, se implementa el componente necesario para el cálculo de la similitud del coseno con las frases incluidas en cada documento de la base de conocimiento. El método encargado de este proceso se detalla en la Figura 11, donde se destacan las funcionalidades de la librería scikit-learn, las cuales facilitan el cálculo de la similitud mediante la vectorización de las frases comparadas.

```python
def verify_user_intent(self, recognizer_input_data: SimilarityBasedInputData) -> SimilarityBasedResponse:
    """
    Determine the user intent based on the user's message and the existing set of phrases.

    :param recognizer_input_data: Object with the data needed for the intent recognizer to get the user intent.
    :return: An object that represents the result of the recognition process.
    """

    # Create the TF-IDF vectorizer
    vectorizer = TfidfVectorizer()

    # Build the TF-IDF array
    tfidf_matrix = vectorizer.fit_transform(
        [recognizer_input_data.user_message] + recognizer_input_data.document_corpus
    )

    # Calculate cosine similarity between sentences
    similarity_matrix = cosine_similarity(tfidf_matrix, tfidf_matrix)

    # Get the max similarity value of the cosine similarities
    max_similarity_value = max(similarity_matrix[0][1:])

    return SimilarityBasedResponse(
        True if max_similarity_value >= recognizer_input_data.confidence_limit else False,
        max_similarity_value
    )
```

Figura 11. Uso de la librería scikit-learn para implementar el motor del reconocimiento de la intención basado en la similitud del coseno.

En esta implementación, se utilizan objetos genéricos para gestionar los parámetros recibidos por el motor de reconocimiento de la intención y los resultados devueltos, promoviendo una estructura escalable.

Adicionalmente, se enfatiza el uso de tipado fuerte en Python, aunque no sea obligatorio para el lenguaje. Esta decisión se respalda con la adopción de PEP 8 para mantener una legibilidad clara en el código, facilitando su comprensión y mantenimiento.

Implementación de la API REST

Se establecen tres rutas cruciales para facilitar un flujo de comunicación efectivo con el reconocimiento de la intención, como se ilustra en la Figura 12. La primera ruta, encargada de procesar los mensajes recibidos, se inicia al recibir un mensaje, filtrándolo a través de una instancia del patrón Proxy incorporado durante la fase de diseño.

Posteriormente, el motor de reconocimiento de la intención entra en acción, identificando el documento más adecuado y permitiendo la generación de una respuesta precisa.

```python
@app.route('/images/<image_name>')
def show_image(image_name):
    """Looks for an image in the static folder."""
    return send_from_directory('static/images', image_name)


@app.route('/api/actions', methods=['POST'])
def get_action_response():
    """Looks for a response based on an action."""
    responses_generator: ActionsResponsesGenerator = ActionsResponsesGenerator()
    actions_proxy: ProxyInterface = ActionsProxy(responses_generator)
    user_request: Request = request

    return actions_proxy.handle_request(user_request)


@app.route('/api/messages', methods=['POST'])
def get_message_response():
    """Looks for a response based on the user's intent and provides an associated response."""
    responses_generator: MessagesResponsesGenerator = MessagesResponsesGenerator()
    messages_proxy: ProxyInterface = MessagesProxy(responses_generator)
    user_request: Request = request

    return messages_proxy.handle_request(user_request)
```

Figura 12. Definición de los endpoints asociados a las funcionalidades cubiertas por el backend del chatbot.

Otra ruta implementada facilita el acceso a respuestas específicas mediante el ID del documento. Esta funcionalidad resulta adecuada para ofrecer recomendaciones o sugerencias, creando así una conexión entre diferentes documentos y enriqueciendo la experiencia del usuario.

La última ruta diseñada permite obtener recursos de imágenes almacenados de forma estática en el backend, proporcionando una forma eficiente de acceder a elementos visuales asociados a los tipos de respuestas que incluyen imágenes.

A continuación, se muestra el listado de los endpoints creados para la interacción con el backend:

- **Endpoint de mensajes:** /api/messages
- **Endpoint de acciones (sugerencias):** /api/actions
- **Endpoint de recursos (imágenes):** /api/images/<image_name>

```python
class MessagesEndpointTestSuite(unittest.TestCase):

    def test_messages_endpoint_status(self):
        url = 'http://127.0.0.1:5000/api/messages'

        data = {'user_message': 'Hola'}
        response = requests.post(url, json=data)

        self.assertEqual(response.status_code, 200)


if __name__ == '__main__':
    unittest.main()
```

Figura 13. Implementación de una prueba unitaria para verificar el funcionamiento del endpoint de mensajes.

En el contexto de la implementación, la Figura 13 detalla la creación de una prueba unitaria específicamente para el endpoint de mensajes. Esta prueba unitaria, elaborada con el objetivo de verificar el funcionamiento del endpoint y su capacidad para manejar futuros cambios, consolida la robustez y la confiabilidad de la funcionalidad implementada.

Implementación de opciones de configuración

Para facilitar la configuración del backend del chatbot, se integra un sistema flexible que utiliza variables de entorno para gestionar las configuraciones fundamentales. Esta práctica permite una adaptabilidad eficiente, proporcionando un control centralizado sobre aspectos clave del sistema. La Figura 14 ilustra la incorporación de estas variables, enfocándose especialmente en dos aspectos esenciales.

En primer lugar, se incluyen variables de entorno para controlar el uso de los componentes de sanitización. Estas variables permiten habilitar o deshabilitar dinámicamente la funcionalidad de sanitización de mensajes, proporcionando una capacidad de ajuste en tiempo real según las necesidades específicas del sistema.

Además, se introducen variables de entorno adicionales para determinar el límite de confianza asociado al reconocimiento de la intención. Estos valores de límite de confianza se establecen en base a la precisión obtenida durante pruebas manuales realizadas hasta el momento.

```
# App settings
HOST=0.0.0.0
PORT=5000

# Requests settings
MAX_USER_MESSAGE_LENGTH=200

# Responses settings
MAX_SUGGESTIONS_NUMBER=2

# Data construction settings
SYNTAX_CLEANER=True
STOP_WORDS_CLEANER=False
LEMMATIZER=False

# Intent recognition settings
CONFIDENCE_LIMIT=0.50
SUGGESTION_LIMIT=0.40
```

Figura 14. Archivo de configuración de las variables de entorno utilizadas en le backend del chatbot.

La Figura 15 muestra el estado del tablero Kanban tras la implementación de la solución propuesta. En este punto, las actividades y tareas incluidas en el tablero se enfocan en la fase de pruebas, evaluación y análisis de resultados.

Figura 15. Actividades y tareas determinadas para la fase de pruebas, evaluación y análisis de resultados.

2.5 Fase de pruebas, evaluación y análisis de resultados

Durante esta fase se lleva a cabo una evaluación integral de la precisión del chatbot, centrada inicialmente en el proceso basado en la similitud del coseno. Esta evaluación se consolida mediante una prueba manual diseñada para medir la eficacia del reconocimiento de la intención. A medida que se avanzaba en esta fase, la aplicación de la prueba se extiende para evaluar diversas propuestas de modelos neuronales considerados para el backend del chatbot.

Los resultados obtenidos proporcionan una visión detallada de la eficacia de cada enfoque, lo que lleva a la inclusión de uno de estos modelos neuronales en el backend del chatbot con el objetivo de mejorar significativamente la precisión en el reconocimiento de intenciones.

Además, como parte del proceso de mejora continua, se automatiza la prueba de rendimiento previamente definida y se realizan algunas mejoras al código implementado hasta el momento.

Definición de pruebas de precisión

Se establece un entorno estructurado para evaluar la precisión del chatbot en el reconocimiento de la intención, centrándose específicamente en el contexto de "reingreso a una carrera". Este ambiente de pruebas se diseña incorporando dos conjuntos de datos clave para guiar el proceso evaluativo.

El primer conjunto de datos utilizados se detalla en la Tabla 6, en la cual se agrupan frases asociadas al contexto específico de "reingreso a una carrera". Estas frases, denominadas "utterances", se consideran elementos cruciales presentes en la base de conocimiento que el chatbot debería reconocer y manejar de manera efectiva.

Tabla 6. Frases incluidas en la base de conocimiento para la evaluación de la precisión del chatbot.

Posibles frases en la base de conocimiento	
Identificador	**Frase**
U1	Proceso de reingreso a una carrera de pregrado
U2	Abandoné la carrera y quiero regresar
U3	Quiero volver a entrar a mi carrera
U4	Me gustaría volver a mi carrera
U5	¿Qué debo hacer para regresar a mi carrera?

Simultáneamente, como lo indica la Tabla 7, se definen tres mensajes que los usuarios podrían enviar al chatbot con el propósito de obtener información sobre el "reingreso a una carrera". Estos mensajes representan situaciones del mundo real y forman la base para las pruebas de reconocimiento de intenciones.

Tabla 7. Mensajes de usuario considerados para la evaluación de la precisión del chatbot.

Posibles mensajes enviados por los usuarios	
Identificador	**Mensaje**
M1	Quiero regresar a mi carrera
M2	¿Cómo puedo volver a la universidad?
M3	Necesito volver a mi carrera

El proceso de prueba implica la comparación de cada uno de estos mensajes de usuario con las frases establecidas en la base de conocimiento. Se determinará el valor de similitud utilizando el modelo base del reconocimiento de la intención, junto con las tres principales técnicas de sanitización: limpieza de sintaxis, eliminación de stopwords y lematización.

Los resultados obtenidos a partir de esta evaluación brindarán una aproximación precisa de la eficacia del chatbot en el reconocimiento de intenciones dentro del contexto específico. Lo cual permitirá evaluar otras posibles opciones de solución como modelos neuronales enfocados en procesos de comparación de oraciones.

Evaluación de la precisión del chatbot

El escenario de prueba se aplica a la implementación actual del reconocimiento de la intención basada en la similitud del coseno, junto con tres procesos de sanitización. Los resultados de similitud obtenidos de los mensajes de usuario en relación con cada frase en la base de conocimiento se presentan detalladamente en la Tabla 8.

Tabla 8. Valores de similitud obtenidos al evaluar el motor del reconocimiento de la intención basado en la similitud del coseno.

	U1	**U2**	**U3**	**U4**	**U5**
M1	0.0560	0.5298	0.5096	0.2491	0.3734
M2	0	0.1822	0.1382	0.1310	0
M3	0.0483	0.0735	0.4393	0.414	0.1686

Estos resultados revelan la necesidad de mejorar la solución para el reconocimiento de la intención. Con este fin, se lleva a cabo una evaluación de varios modelos neuronales basados en Transformers, obteniendo los resultados detallados en el Anexo IV. La evaluación comparativa de estos modelos permite la identificación y selección del modelo “paraphrase-multilingual-MiniLM-L12-v2” basado en el framework SentenceTransformer, el cual se encuentra disponible bajo la licencia Apache 2.0 en [16]. Este modelo se implementará con el objetivo de incrementar significativamente la precisión del reconocimiento de la intención en el backend del chatbot.

Incorporación de un modelo refinado

Para mejorar la precisión del chatbot, se lleva a cabo la implementación del modelo seleccionado para Sentence Similarity en el backend del chatbot, aprovechando eficientemente el patrón Factory Method previamente incorporado. La introducción de este nuevo modelo simplemente requiere la creación de una fábrica asociada a la lógica del nuevo enfoque.

```python
def verify_user_intent(self, recognizer_input_data: SimilarityBasedInputData) -> SimilarityBasedResponse:
    """
    Determine the user intent based on the similarity of the user message and the current corpus.

    :param recognizer_input_data: Object with the data needed for the intent recognizer to get the user intent.
    :return: An object that represents the result of the recognition process.
    """

    # Limit the number of utterances analyzed
    if int(os.getenv('UTTERANCES_LIMIT')):
        document_corpus = random.sample(
            recognizer_input_data.document_corpus,
            min(int(os.getenv('UTTERANCES_LIMIT')), len(recognizer_input_data.document_corpus))
        )
    else:
        document_corpus = recognizer_input_data.document_corpus

    # Get the embeddings representations of the corpus and user message
    corpus_embeddings = self._model.encode(
        document_corpus,
        convert_to_tensor=True
    )
    user_message_embeddings = self._model.encode(
        [recognizer_input_data.user_message],
        convert_to_tensor=True
    )

    # Compute cosine similarities
    cosine_similarities = util.cos_sim(corpus_embeddings, user_message_embeddings)

    # Get the max similarity value of the cosine similarities
    max_similarity_value = max(cosine_similarities).item()

    return SimilarityBasedResponse(
        True if max_similarity_value >= recognizer_input_data.confidence_limit else False,
        max_similarity_value
    )
```

Figura 16. Incorporación de un modelo basado en Transformers para mejorar la precisión del motor del reconocimiento de la intención.

Así pues, como se evidencia en la Figura 16, ahora la lógica para el reconocimiento de la intención utiliza este modelo para obtener representaciones vectoriales mejoradas de los mensajes de los usuarios las frases de los documentos.

A través del cálculo de la similitud del coseno, se obtiene un valor de similitud más preciso, gracias a la calidad mejorada de las representaciones vectoriales proporcionadas por el nuevo modelo implementado.

Dada la considerable demanda de recursos de hardware asociada al uso del modelo, se implementa el patrón Singleton junto con la fábrica del modelo, dando lugar a una arquitectura optimizada, destacada en la Figura 17. La adopción de estos patrones de diseño contribuye significativamente a la escalabilidad del sistema, gestionando eficientemente los recursos necesarios.

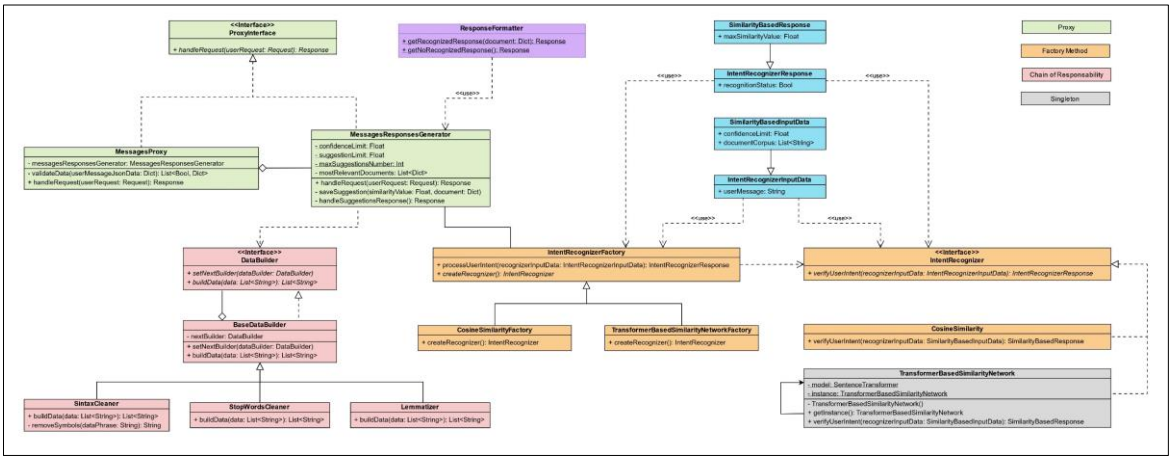

Figura 17. Diagrama de clases del módulo del reconocimiento de la intención después de incorporar un modelo basado en Transformers.

A pesar de que el modelo base cuenta con una precisión adecuada, se realiza un esfuerzo adicional para afinarlo y así mejorar aún más el reconocimiento de la intención. Siguiendo los pasos detallados en la guía del Anexo V, se llevan a cabo actividades específicas para preparar los datos de entrenamiento utilizados en este proceso de afinamiento.

Finalmente, para asegurar la constante mejora del chatbot, se automatiza la prueba de precisión previamente definida, permitiendo una verificación sencilla de la efectividad del entrenamiento. La Figura 18 presenta uno de los resultados obtenidos tras realizar el proceso de evaluación en el modelo afinado. Se observa que el escenario de prueba automatizado ahora permite evaluar el porcentaje de precisión del chatbot al identificar la

intención de los usuarios, y el porcentaje de reducción de errores que se consiguió tras afinar el modelo utilizado.

```
----------------------------------------------------------------------
Ran 2 tests in 15.360s

OK

PERFORMANCE IMPROVEMENT TEST:
Base model performance: 0.90744
Fined tuned model performance: 0.97365
Performance improvement: 6.62%

ERROR REDUCTION TEST:
Base model wrong similarity: 0.38667
Fined tuned model wrong similarity: 0.03743
Error reduction: 34.92%
```

Figura 18. Resultados finales de la evaluación automatizada para determinar la precisión del módulo del reconocimiento de la intención.

Estos resultados demuestran que el chatbot ha alcanzado una precisión notable al reconocer las intenciones de los usuarios, especialmente en temas relacionados con los documentos de la base de conocimiento. Este refinamiento del modelo ha contribuido significativamente a la efectividad general del chatbot en la interpretación precisa de las intenciones de los usuarios.

Prueba manual con el modelo refinado

Nuevamente, el escenario de prueba anteriormente definido se aplica a la implementación actual del reconocimiento de la intención basada en un modelo refinado. Los resultados de esta prueba se muestran en la Tabla 9, en la cual se observa que el módulo de reconocimiento de la intención ahora es capaz de establecer mejores relaciones semánticas en el contexto de la información utilizada en la FIS.

Tabla 9. Valores de similitud obtenidos al evaluar el motor del reconocimiento de la intención basado en un modelo refinado.

	U1	U2	U3	U4	U5
M1	0.9218	0.9417	0.9540	0.9815	0.9773
M2	0.8054	0.8459	0.8395	0.7998	0.7651
M3	0.9795	0.9016	0.8998	0.9501	0.9506

Refactorización del backend

Se lleva a cabo un proceso de mejora del código fuente, centrado en garantizar la conformidad con el estándar PEP 8, ampliamente reconocido en la comunidad de Python. Para realizar esta tarea de manera eficiente, se emplea la librería pycodestyle, la cual analiza el código existente y señala todos los errores con respecto a las directrices establecidas por PEP 8.

Adicionalmente, como parte del esfuerzo por mantener un código claro y de fácil comprensión, se realizan cambios en la nomenclatura de algunas variables. Este proceso de renombramiento se lleva a cabo con el objetivo de dotar al código de mayor legibilidad y coherencia, facilitando así la comprensión y el mantenimiento continuo del backend.

En este punto, la Figura 19 indica el estado del tablero Kanban tras la fase de pruebas, evaluación y análisis de resultados. Así pues, el tablero incluye las actividades y tareas que se esperan realizar durante la fase de integración.

Figura 19. Actividades y tareas determinadas para la fase de integración.

2.6 Fase de integración

En la fase de integración, el backend del chatbot se implementa en un entorno virtualizado proporcionado por la Facultad de Sistemas. La Facultad realiza pruebas de seguridad para evaluar y garantizar la robustez del servicio. Estas pruebas también se utilizan para establecer medidas de acceso que aseguran la seguridad y la integridad del servicio implementado. Este enfoque integral demuestra el compromiso de la Facultad con la calidad y la seguridad en el despliegue del chatbot.

Levantamiento e integración del backend

El backend del chatbot levanta utilizando el servidor WSGI llamado Waitress, diseñado especialmente para entornos Windows. Este servidor WSGI proporciona una plataforma robusta para ejecutar aplicaciones web en este entorno operativo específico.

Gracias a la utilización de Waitress, se logra un despliegue eficiente del backend, permitiendo un acceso fluido a su documentación y funcionalidades. La Figura 20 muestra el estado del backend tras realizar el proceso de levantamiento.

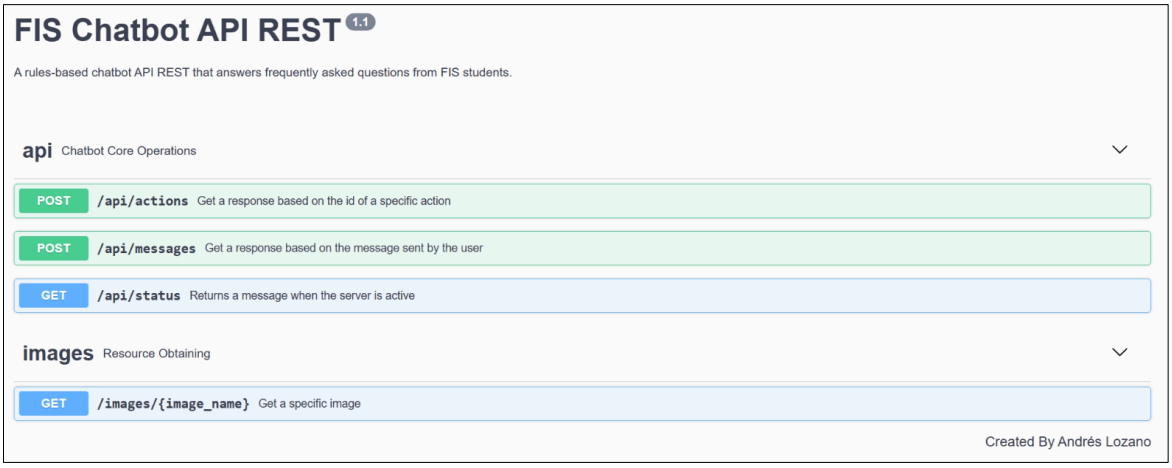

Figura 20. Estado del backend del chatbot tras el proceso de levantamiento en el VDI de la Facultad de Ingeniería de Sistemas.

La integración del backend del chatbot se lleva a cabo uniendo eficientemente sus funcionalidades con los demás componentes del proyecto. Esta integración ha resultado en un servicio integral que ahora puede ser utilizado de manera conjunta con los demás elementos del sistema.

Validación de la integración

En la etapa final de la integración del componente, se lleva a cabo una evaluación de seguridad por parte de la Facultad de Ingeniería de Sistemas (FIS), la cual no revela problemas significativos asociados al backend del chatbot. Como resultado de esta evaluación, se recibieron sugerencias para fortalecer la seguridad, entre las cuales se implementa el cierre de puertos innecesarios en el entorno virtualizado (VDI).

Esta medida de seguridad adicional contribuye a garantizar la integridad y confidencialidad del sistema. Con estos resultados, se concluye exitosamente la integración del backend con los demás componentes del proyecto, asegurando un entorno seguro y funcional para su implementación conjunta.

3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES

En esta sección, se presentan los logros obtenidos durante las fases finales del desarrollo del backend del chatbot. Los resultados destacan la implementación de la API REST, que

brinda respuestas precisas mediante la similitud de oraciones y demuestra eficiencia frente a cargas concurrentes.

En la sección de conclusiones, se establece una relación integral entre los resultados alcanzados y los objetivos específicos propuestos al inicio del proyecto. Se analiza cómo la solución desarrollada cumple con la efectividad, eficiencia y personalización esperadas para la atención de procesos académicos.

Finalmente, en las recomendaciones, se exploran escenarios adicionales que podrían fortalecer y complementar aún más el componente del backend del chatbot en el futuro.

3.1 Resultados

A continuación, se presenta el estado final de la API REST del chatbot después del proceso de integración. A través de este análisis, se busca demostrar el cumplimiento de los objetivos específicos propuestos. Además, se compartirán los resultados obtenidos de las pruebas de carga realizadas, evaluando así la eficiencia y capacidad del sistema ante un flujo significativo de solicitudes.

Estado de los endpoints de la API REST

La implementación de la API REST representa un logro clave en la consecución de los objetivos planteados para el backend del chatbot. En este sentido, la finalización de este componente se ha logrado mediante el levantamiento del backend en el entorno proporcionado por la Facultad de Ingeniería de Sistemas, lo que ha permitido su integración con los demás componentes del proyecto.

Así pues, la Figura 21 refleja la respuesta del backend al recibir un mensaje mediante la estructura de datos diseñada en la fase inicial. Esta respuesta valida la efectividad del reconocimiento de la intención, proporcionando una respuesta precisa y personalizada que incluye mensajes de texto, enlaces y sugerencias hacia otros documentos.

Request URL

http://127.0.0.1:5000/api/messages

Server response

Code	Details
200	Response body


```
[
  {
    "content": "La \"Matrícula Ordinaria\" se realiza antes del inicio de clases según las fechas
actividades:",
    "type": "text"
  },
  {
    "alt": "Actividades relacionadas a la \"Matrícula Ordinaria\"",
    "path": "ordinary_registration_activities.png",
    "type": "image"
  },
  {
    "content": "Puedes revisar el Calendario Académico en el siguiente enlace:",
    "type": "text"
  },
```


Figura 21. Respuesta obtenida al utilizar el endpoint de mensajes de la API REST.

Por su parte, la Figura 22 evidencia la capacidad del backend para responder a solicitudes específicas de documentos, permitiendo el acceso directo a información relevante almacenada en la base de conocimiento.

Request URL

http://127.0.0.1:5000/api/actions

Server response

Code	Details
200	Response body


```
[
  {
    "content": "Durante este periodo académico existen tres tipos de matrículas disponibles: \"Matrícula Ordinaria\", \"Matrícula
Especial\".",
    "type": "text"
  },
  {
    "content": "Si necesitas conocer más detalles sobre algún tipo de matrícula puedes hacerlo mediante los siguientes mensajes:",
    "type": "text"
  },
  {
    "action": "matricula-ordinaria",
    "name": "Matrícula Ordinaria",
    "type": "action"
  },
```


Figura 22. Respuesta obtenida al utilizar el endpoint de acciones de la API REST.

En este mismo contexto, la Figura 23 demuestra la capacidad del chatbot para procesar solicitudes de recursos de imagen, proporcionando respuestas visuales que complementan la información descrita en los mensajes de tipo texto.

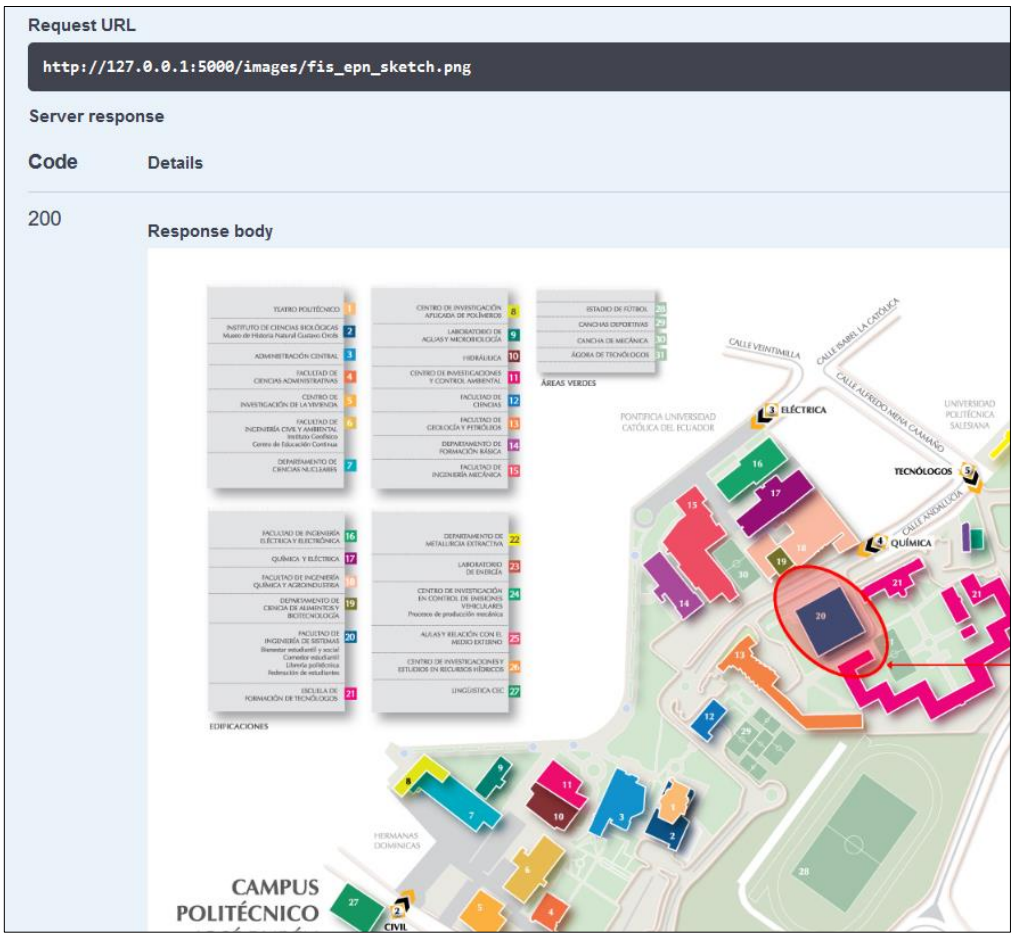

Figura 23. Respuesta obtenida al utilizar el endpoint de imágenes de la API REST.

Finalmente, la Figura 24 valida el estado general del backend, confirmando su disponibilidad y capacidad para procesar solicitudes en cualquiera de los endpoints descritos.

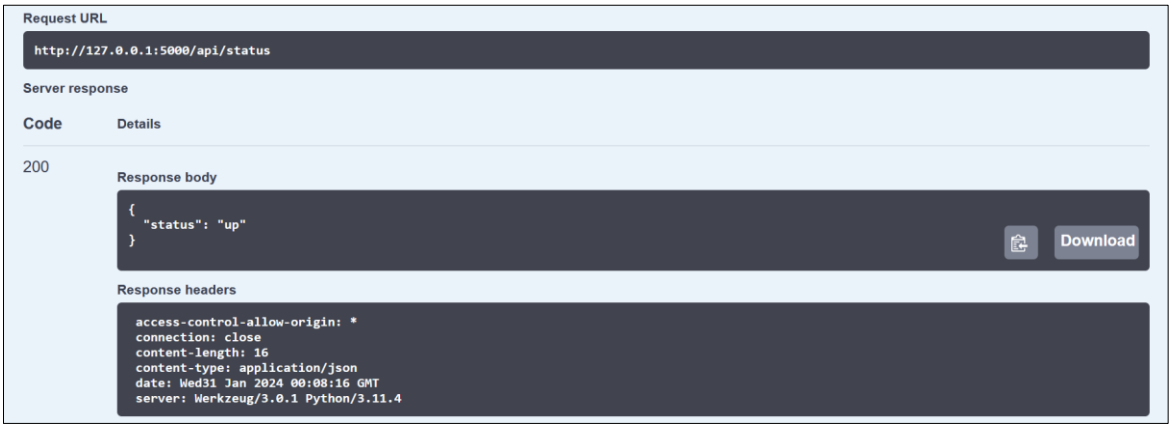

Figura 24. Respuesta obtenida al utilizar el endpoint de estado de la API REST.

Este conjunto de resultados confirma la efectividad de la solución implementada, cumpliendo con el objetivo de ofrecer un servicio operativo que cumpla con las características seleccionadas para el backend del chatbot.

Así pues, la solución implementada no solo logra reconocer con una buena precisión las intenciones de los usuarios, sino que también facilita el acceso a información específica y recursos multimedia, respaldando así el objetivo general del componente.

Carga soportada por el backend

La evaluación de la capacidad de carga del backend del chatbot se llevó a cabo mediante una prueba automatizada utilizando la herramienta Locust para Python. La configuración de la prueba se diseñó para simular un escenario realista con un total de 20 usuarios concurrentes, generando un promedio de 13 peticiones por segundo. En este entorno, el servidor del backend se inició con 16 hilos, cada uno encargado de gestionar el conjunto de peticiones concurrentes.

Los resultados presentados en la Figura 25, revelan que, con esta configuración, el tiempo de respuesta para las peticiones asociadas a los mensajes de los usuarios alcanzó un promedio de 0.2 segundos.

Este valor, considerando la presencia de 20 usuarios concurrentes, se mantiene dentro de límites aceptables en comparación con los servicios de chatbot evaluados, indicando una capacidad de respuesta adecuada del backend bajo una carga media.

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	/api/actions	10	12	14	15	17	22	29	67
POST	/api/messages	96	140	200	280	530	750	1300	1900
GET	/api/status	4	4	5	5	7	10	20	53
GET	/images/fis_epn_sketch.png	13	16	18	20	24	31	45	110
	Aggregated	**12**	**16**	**21**	**58**	**140**	**280**	**820**	**1900**

Figura 25. Resultados generales de la prueba de carga realizada en el ambiente del VDI.

Al analizar la Figura 26, se observa que este tiempo de respuesta se mantiene constante (promedio de 0.013 segundos por petición) incluso cuando la carga sobre el servidor varía ligeramente. Estos resultados confirman la estabilidad del backend, sugiriendo que la configuración actual en el ambiente del VDI es óptima para manejar el nivel de carga establecido durante la prueba.

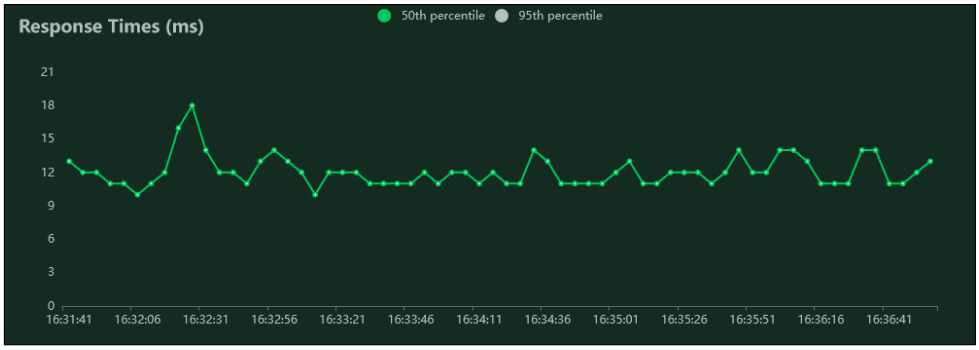

Figura 26. Resultados de los tiempos de respuesta del backend del chatbot.

En términos de estabilidad y rendimiento, la Figura 27 demuestra que, a lo largo de la carga configurada, no se produjeron errores ni caídas del servidor. Este aspecto destaca la robustez del backend del chatbot, mostrando que mantiene su efectividad incluso ante variaciones significativas en las consultas por segundo.

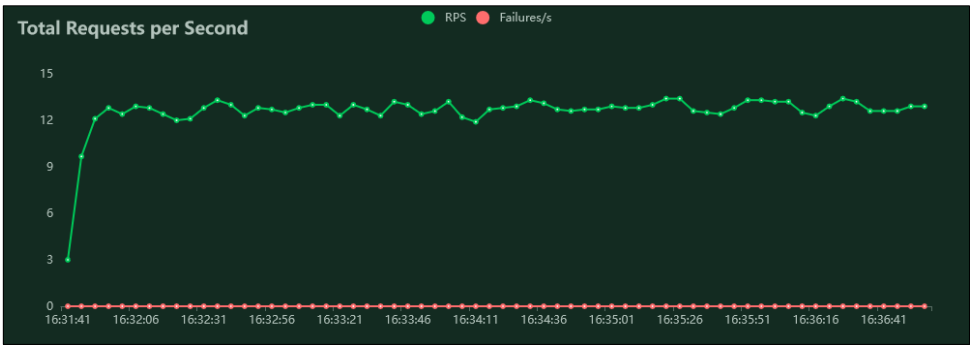

Figura 27. Resultados de las peticiones y fallas del backend del chatbot.

En conjunto, estos resultados respaldan la capacidad del backend para manejar cargas sustanciales, garantizando una experiencia de usuario fluida y sin interrupciones, incluso en situaciones de alta demanda para la FIS.

3.2 Conclusiones

La evaluación de las herramientas disponibles en la industria permitió seleccionar las mejores alternativas para satisfacer las necesidades de la Facultad de Ingeniería de Sistemas. Este proceso aseguró la identificación de arquitecturas, funcionamientos y tecnologías subyacentes relevantes, brindando una base sólida para la construcción de una solución efectiva.

El contraste entre las herramientas identificadas y los requerimientos específicos de la facultad resultó en la determinación de características relevantes y adecuadas para la atención de procesos académicos. Esta fase de análisis aseguró que la solución propuesta estuviera alineada con los objetivos particulares de la institución, garantizando una respuesta precisa y enfocada a sus necesidades.

La fase de diseño de la arquitectura del backend, respaldada por diagramas UML detallados, proporcionó una representación visual clara y estructurada de la interacción entre los diversos componentes. La implementación estratégica de patrones de diseño específicos no solo garantizó una eficiencia operativa sobresaliente, sino que también sentó las bases para una escalabilidad significativa. Estos patrones aseguraron una ejecución robusta y adaptativa, proporcionando una estructura flexible capaz de afrontar con éxito futuros cambios y expansiones del sistema.

La implementación exitosa de la API REST para el reconocimiento de la intención no solo fue consistente con el objetivo general del proyecto, sino que también se destacó como un componente crucial para facilitar la comunicación efectiva entre diversos elementos, especialmente con el frontend del chatbot. Este mecanismo demostró ser eficaz y versátil, asegurando una integración sin problemas entre los componentes del sistema. Las pruebas de precisión respaldaron la capacidad de la API para proporcionar respuestas precisas a las solicitudes de los usuarios, consolidando así su papel central en el logro de los objetivos específicos del proyecto.

La solución desarrollada no solo ha demostrado ser efectiva, proporcionando respuestas precisas, sino que también se ha revelado como eficiente, soportando cargas considerables durante las pruebas. La personalización se ha logrado mediante la implementación de patrones de diseño que permiten la escalabilidad de la arquitectura y la adaptación del modelo de respuesta, asegurando la flexibilidad necesaria para futuros desarrollos y expansiones del chatbot. En conjunto, la solución brinda una herramienta sólida y adaptativa para la atención de procesos académicos de grado en la Facultad de Ingeniería de Sistemas.

3.3 Recomendaciones

Durante el proyecto, se evidenció la necesidad de implementar un control de versionamiento para los modelos neuronales basados en Transformers utilizados en el reconocimiento de la intención. Sería altamente recomendable incorporar un sistema que

permita mantener el historial de los modelos entrenados y el estado correspondiente de la base de conocimiento. Esto facilitaría la gestión de cambios, la reversión a versiones anteriores y la trazabilidad del rendimiento del chatbot a lo largo del tiempo.

La automatización del proceso de entrenamiento es esencial para mejorar la eficiencia y reducir la carga manual asociada. Por esta razón, se recomienda desarrollar un sistema automatizado que permita la generación de archivos de entrenamiento de manera sencilla. Esto no solo optimizará el tiempo empleado en la preparación de los datos, sino que también garantizará consistencia y precisión en la selección de ejemplos de entrenamiento, mejorando así la calidad del modelo.

Aunque el ambiente del VDI ha demostrado ser adecuado para niveles de carga intermedios, sería recomendable evaluar y ajustar la arquitectura de hardware o la configuración del módulo de reconocimiento de la intención si se espera un aumento significativo en la carga del chatbot. Este ajuste garantizará una eficiencia continua del servicio, incluso con la incorporación de nuevas funcionalidades o un incremento en el volumen de preguntas.

Con el fin de fortalecer la estructura y reducir dependencias externas, se sugiere que el backend del chatbot se integre de manera más profunda con el componente de persistencia. Esto no solo consolidará la arquitectura, sino que también simplificará la gestión y el mantenimiento del sistema en general. Una integración más estrecha puede contribuir a una mayor cohesión y modularidad, facilitando futuras actualizaciones y expansiones del chatbot.

4 REFERENCIAS BIBLIOGRÁFICAS

[1] D. V.-C. R. V. M.-M. Pablo Ramírez-Hernández, «Revisión de los chatbots basados en inteligencia artificial en la administración pública: Hacia una arquitectura para el gobierno,» *Espacios Públicos,* vol. 24, nº 60, p. 21, 2023.

[2] M. A. V.-G. J. A. R.-O. R. G.-R. Beatriz A. González-Beltrán, «Una revisión de chatbots en la salud,» *Research in Computing Science,* vol. 151, nº 8, p. 120, 2022.

[3] H. D. C.-V. F. C. C.-M. Luis A. Pachas-Santos, «Chatbot basado en el aprendizaje profundo para recomendar productos relevantes,» *Computación y Sistemas,* vol. 27, nº 2, pp. 511-523, 2023.

[4] S. S. F. Sacoto, «Diseño y desarrollo de un chatbot usando redes neuronales recurrentes y procesamiento de lenguaje natural para tiendas virtuales en comercio electrónico,» Universidad Politécnica Salesiana, Cuenca, Ecuador, 2021.

[5] M. Fong Juárez, «Similitud Semántica Textual para Asistentes Virtuales,» Benemérita Universidad Autónoma, Puebla, 2021.

[6] K. I. Y. T. C. M. Cordero Mena, «Diseño y desarrollo de un chatbot usando redes neuronales y procesamiento de lenguaje natural orientado a entidades bancarias,» Universidad Politécnica Salesiana, Cuenca, Ecuador, 2022.

[7] A. K. N. M. Ilieva R., «A Cynefin Framework for Agile Decision Making of AI BOTS,» *International Conference on High Technology for Sustainable Development (HiTech),* pp. 1-4, 2018.

[8] M. g. &. A. D. &. S. K. I. &. O. A. &. P. P. &. B. H. Hassan, «A framework for implementing a Supplier Kanban System through an action research methodology,» vol. 30, pp. 1562-1587, 2022.

[9] V. &. B. K. &. R. G. González Jaramillo, «Aplicación de la metodología de Notación y Modelado de Procesos de Negocios (BPMN) y propuesta de técnica Lean para la Mejora de Procesos Administrativos de una Empresa,» *Global Partnerships for Development and Engineering Education,* nº 14, 2017.

[10] K. S. Martin Fowler, UML gota a gota, Juárez, México, 1999.

[11] R. H. R. J. J. V. Erich Gamma, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.

[12] E. B. Alzate, «Implementación de pruebas unitarias,» Universidad de Antioquia, 2021.

[13] B. W. N. C. Guido van Rossum, «Style Guide for Python Code (PEP 8),» 2001.

[14] M. Grinberg, Flask Web Development, Sebastopo, CA: O'Reilly, 2018.

[15] A. Géron, Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para construir sistemas inteligentes, O'Reilly, 2020.

[16] N. a. G. I. Reimers, «HuggingFace,» Association for Computational Linguistics, 11 2019. [En línea]. Available: https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. [Último acceso: 28 Julio 2023].

5 ANEXOS

ANEXO I. Encuesta de Factibilidad del Proyecto

ANEXO II. Criterios de Evaluación de las Herramientas Chatbot

ANEXO III. Resultados de la Evaluación de los Servicios de Chatbot

ANEXO IV. Resultados de la Evaluación de Precisión del Reconocimiento de la Intención

ANEXO V. Guía para el Entrenamiento del Modelo del Chatbot

ANEXO I

Encuesta de Factibilidad del Proyecto:

Para evaluar la factibilidad del proyecto, se realizó una encuesta con el objetivo de analizar los tiempos de atención brindados a las consultas de los estudiantes, especialmente de aquellas realizadas mediante correo electrónico. Los resultados más relevantes para este caso de estudio se presentan a continuación:

La Figura 28 muestra los tiempos que los estudiantes han experimentado desde que realizan una consulta mediante correo electrónico hasta obtener una respuesta. En términos generales, se ha determinado que el tiempo de respuesta promedio es de aproximadamente 48 horas.

Este indicador ofrece una visión clara de la eficiencia actual en la gestión de consultas electrónicas por parte del personal administrativo. Así pues, se considera que el tiempo de atención es considerablemente excesivo.

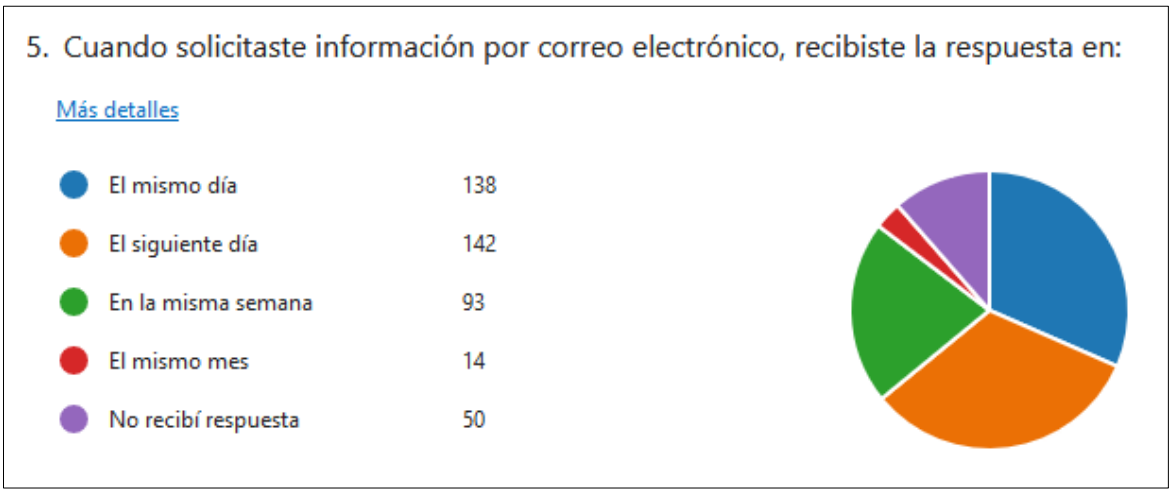

Figura 28. Tiempos de respuesta a consultas por correo electrónico.

La Figura 29 presenta la frecuencia con la que los estudiantes realizan consultas mediante correos electrónicos, centrándose especialmente en periodos de matrícula. Extrapolando los resultados obtenidos junto con los comentarios del personal administrativo de la FIS, se ha identificado que, en periodos de baja demanda, los estudiantes suelen realizar aproximadamente una consulta diaria. En contraste, durante periodos de alta demanda, los

estudiantes incrementan su frecuencia de consultas, llegando a realizar hasta un promedio de tres consultas diarias.

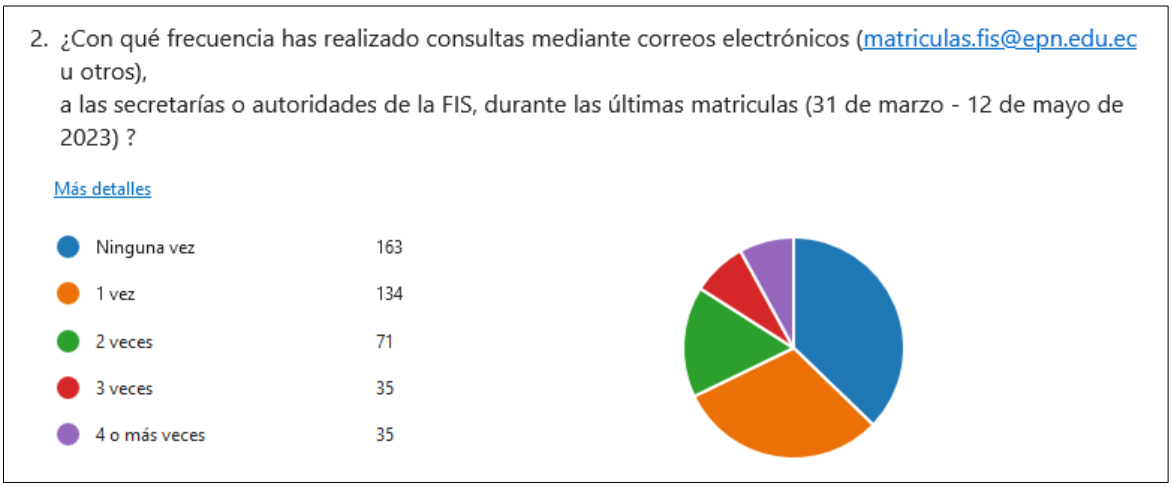

Figura 29. Frecuencia de consultas por correo electrónico en periodos de alta demanda.

Estos resultados ofrecen una perspectiva detallada de los patrones de interacción de los estudiantes con el sistema de consultas y destacan la necesidad de implementar estrategias que optimicen los tiempos de respuesta, especialmente en épocas de alta demanda.

ANEXO II

Criterios de Evaluación de las Herramientas de Chatbot:

Enlace al repositorio que contiene el documento:

https://epnecuador-my.sharepoint.com/:x:/g/personal/andres_lozano_epn_edu_ec/EUcvJh1igaVNmVR-ELqPfkEBo4kySa8dHCyV4j1KvwYfkg?e=lxay38

ANEXO III

Resultados de la Evaluación de los Servicios de Chatbot:

Enlace al repositorio que contiene el documento:

https://epnecuador-my.sharepoint.com/:x:/g/personal/andres_lozano_epn_edu_ec/EbpBUKgJ6mlAqmwCFdRwRfEB6xtkPwF0ps_QkY1sK3qZzw?e=hSuxjJ

ANEXO IV

Resultados de la Evaluación de Precisión del Reconocimiento de la Intención:

Enlace al repositorio que contiene el documento:

https://epnecuador-my.sharepoint.com/:x:/g/personal/andres_lozano_epn_edu_ec/EQ6yWDu5LldOlvRhb2nX-20B5RbURN6G6TEeud9dKn6eMQ?e=uphwup

ANEXO V

Guía para el entrenamiento del modelo del chatbot:

El funcionamiento eficiente del chatbot se basa en la capacidad de reconocer las intenciones de los usuarios, lo cual se logra comparando los mensajes recibidos con las "utterances" almacenadas en la base de conocimiento. En vista de la necesidad constante de adaptar el chatbot a nuevas preguntas formuladas por los estudiantes, resulta fundamental perfeccionar el modelo mediante la incorporación de información actualizada.

Así pues, para garantizar un reconocimiento de la intención más preciso, está guía proporciona una descripción detallada de los pasos a seguir para afinar el modelo con las nuevas preguntas que se pretenden agregar a la base de conocimiento.

Directorio del modelo del chatbot:

Una vez concluido el desarrollo de este componente, el chatbot entregado cuenta con un modelo refinado que se encuentra almacenado en el repositorio de modelos de Hugging Face. Para integrar este modelo al sistema, es esencial especificar su nombre en la propiedad *FINED_TUNED_MODEL* del archivo de variables de entorno. En caso de que el modelo sea privado, será necesario proporcionar un token, el cual se debe colocar en la propiedad *MODEL_TOKEN*. Se recomienda encarecidamente alojar los modelos en este repositorio, ya que facilita significativamente el proceso de implementación del chatbot.

Además, se brinda la opción de guardar el modelo del chatbot en un directorio denominado *models/*, ubicado en la raíz del proyecto. La aplicación está diseñada para priorizar la búsqueda de modelos en este directorio, y en caso de no encontrarlos, intentará descargarlos automáticamente desde el repositorio.

Tras completar el proceso de entrenamiento, el modelo se almacenará automáticamente en el directorio *models/*, asegurando que el chatbot tenga acceso prioritario a esta versión local durante su funcionamiento.

Adición de nuevas preguntas en el backend:

Para entrenar un modelo de chatbot se deben colocar los nuevos documentos de forma manual en el backend. Para ello, se deben seguir los siguientes pasos:

1. Crear los nuevos documentos que se agregarán en el chatbot siguiendo la estructura de documentos descrita en la Figura 7. Se recomienda colocar al menos

diez “utterances” por cada documento para mejorar la precisión del chatbot al reconocer una misma intención por medio de diferentes frases.

2. El documento static/documents/documents.py es el archivo principal desde el cual el chatbot leerá todos los documentos ingresados para el proceso de entrenamiento. Por lo cual, todos los documentos que se hayan creado deben ser importados en el archivo mencionado tal como se lo ilustra en la Figura N.

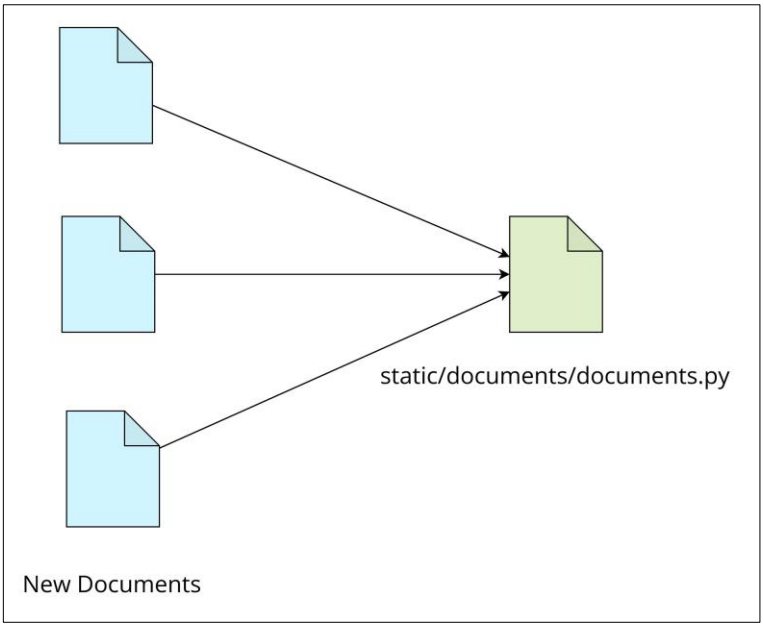

Figura 30. Esquema para la creación manual de nuevos documentos en el backend del chatbot.

Preparación de los datos de entrenamiento:

Cuando los nuevos documentos se hayan ingresado en el backend del chatbot según el esquema planteado, se deben crear un archivo con los datos de entrenamiento que utilizará el chatbot para el proceso de aprendizaje. Así pues, los datos de entrenamiento deben generarse siguiendo los siguientes pasos:

1. Por cada nuevo documento ingresado, se recomienda seleccionar al menos 2 “utterances”, las cuales serán emparejadas con otras “utterances” de los demás documentos existentes.

   Por ejemplo, supongamos que el chatbot cuenta con un solo documento y se quiere agregar uno más. De cada uno de los documentos existentes, se seleccionan 2 “utterances” y se las agrupa tal como lo muestra la Figura 31.

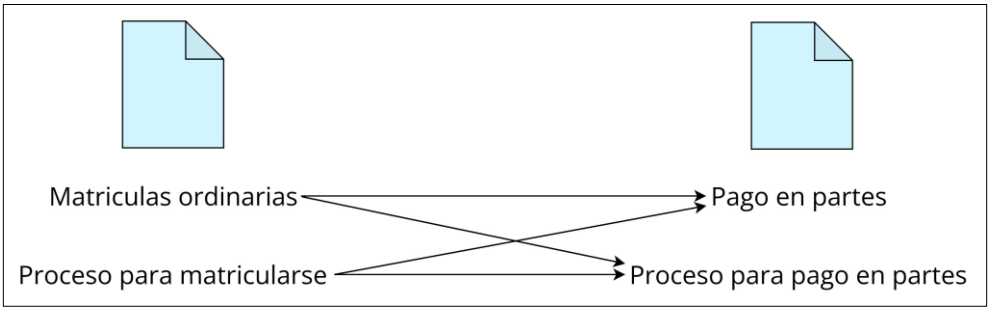

Figura 31. Esquema de relación de documentos para el entrenamiento.

A partir de esta agrupación se puede obtener una tabla similar a la presentada en la Tabla N, en la cual se utilizan dos columnas para establecer la relación entre las "utterances" seleccionadas, y una tercera columna para asignar un valor de similitud entre ambas frases.

Tabla 10. Ejemplo de conjunto de datos para el entrenamiento del modelo.

Utterances 1	Utterances 2	Similitud
Matriculas ordinarias	Pago en partes	0.1
Matriculas ordinarias	Proceso para pago en partes	0
Proceso para matricularse	Pago en partes	0.15
Proceso para matricularse	Proceso para pago en partes	0.1

2. Por cada una de las relaciones de la tabla se debe asignar manualmente un valor de similitud. Este valor de similitud debe representar la relación semántica que existe entre las "utterances" seleccionadas. Es importante relacionar todas las "utterances" de todos los documentos para que el chatbot aprenda a diferenciar los documentos que se relacionan y los que no lo hacen.

3. La tabla resultante tras asignar los valores de similitud se debe guardar en el directorio *training/examples/fis-examples.csv* manteniendo el formato del archivo.

Entrenamiento del modelo:

Para el entrenamiento del modelo se ha creado un script de Python denominado *trainer.py*, el cual contiene toda la lógica necesaria para generar un nuevo modelo a partir de los datos de entrenamiento creados y los documentos existentes.

Los parámetros de entrenamiento del modelo se pueden ajustar mediante las variables de entorno *BATCH_SIZE* y *EPOCHS* según las características necesarias.

Así pues, para generar un nuevo modelo entrenado se deben seguir los siguientes pasos:

1. Asegurarse de que en el archivo *trainer.py* se haga referencia al documento de entrenamiento previamente generado. Además, se recomienda crear el directorio *models/* para que posteriormente se guarde el nuevo modelo.
2. Ejecutar el script de entrenamiento *trainer.py* y ver el modelo resultante en el directorio */models*.

Nota: Generalmente el entrenamiento tarde entre una y dos horas, por lo cual se recomienda utilizar un dispositivo potente. Preferentemente, se debe realizar el proceso de entrenamiento en un computador con tarjeta gráfica.

Finalización del entrenamiento:

Al finalizar el entrenamiento el directorio *models/* contendrá el nuevo modelo afinado a partir de los nuevos documentos ingresados. Tras este proceso se debe acudir a las guías del componente de persistencia para ingresar los datos de los nuevos documentos en la base de datos.